

Mitigation of Coning and Motion Blur Effects in Star Trackers

Master Thesis



Mitigation of Coning and Motion Blur Effects in Star Trackers

Master Thesis
February, 2026

By
Filip Penzar

Copyright: Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.

Cover photo: Vibeke Hempler, 2012

Published by: DTU, Department of Space Research and Technology, Elektrovej, Buildings 327+328, 2800 Kgs. Lyngby Denmark
www.space.dtu.dk

Approval

This thesis has been prepared at the Section for Measurement and Instrumentation Systems, Department of Space Research and Space Technology, at the Technical University of Denmark, DTU, in partial fulfillment for the degree Master of Science in Autonomous Systems, MSc Eng.

It is assumed that the reader has a basic knowledge in the areas of linear algebra, pinhole camera model, image and signal processing.

Filip Penzar - s232452



.....
Signature

22/02/2026

.....
Date

Abstract

Accurate spacecraft attitude determination is required for a wide range of space missions. Star trackers provide highly precise absolute attitude estimates but suffer performance degradation during rapid rotational motion due to image blur. Gyroscopes complement star trackers by providing high rate angular velocity measurements, but their integration is prone to errors, particularly in the presence of coning motion caused by the non-commutativity of finite rotations. This thesis investigates two related problems. First, the effect of rotational motion on star tracker images is modeled, and image deblurring and denoising techniques are applied to mitigate motion induced degradation. The image restoration methods are evaluated on a representative star field image. Second, coning error compensation methods for gyroscope attitude propagation are analyzed and evaluated using both simulation and experimental data from the DTU star tracker. The results show that coning errors are largely negligible within the operational angular rate range of the star tracker when attitude updates are performed at sufficiently high frequency. The image restoration results show that denoising and deblurring improve centroiding accuracy and increase the number of detectable stars in degraded images.

Acknowledgements

Mathias Benn, Associate Professor, Department of Space Research and Technology Measurement and Instrumentation Systems, Supervisor

I would like to thank my supervisor, Mathias, for his continuous support and guidance throughout this project. I especially appreciate his thoughtful feedback and willingness to help during challenging stages, as well as the after hours data collection in cold weather.

Stevan

Thank you to Stevan for taking the time to proofread the thesis and for the honest, constructive feedback. It made the final version clearer and more readable.

Family and Friends

A big thank you to my family and friends for the constant support, patience, and encouragement. Thanks for listening, for understanding the busy periods, and for helping me relax when it was needed most.

Contents

Preface	ii
Abstract	iii
Acknowledgements	iv
1 Introduction	1
2 Mathematical Representations of Spacecraft Attitude	3
2.1 Reference Frames	3
2.2 Attitude Representations	4
3 Attitude Sensors	9
3.1 Gyroscopes	9
3.2 Star Trackers	10
4 Star Tracker Image Modeling and Deblurring	13
4.1 Star Image Formation	13
4.2 Image Degradation Model	14
4.3 Image Denoising	16
4.4 Image Deblurring Methods	17
5 Attitude Kinematics	21
5.1 Coning Motion	21
5.2 Attitude Propagation Methods	23
6 Star Tracker Image Restoration Results	27
6.1 Methodology	27
6.2 Results	27
6.3 Discussion	31
7 Coning Motion Analysis and Validation	33
7.1 Methodology	33
7.2 Simulation	34
7.3 Experimental Results	38
7.4 Discussion	43
8 Conclusion and Future Work	45
Bibliography	47
A Conversions Between Attitude Representations	49
B Image Restoration Additional Results	51
C Coning Simulation Results	53
D Coning Experiment Results	57
E Use of Generative AI	61

1 Introduction

Spacecraft operating in orbit must be able to accurately determine and control their orientation in space, in order to perform mission critical tasks such as Earth observation, communication, rendezvous and docking, and scientific measurements. This orientation, referred to as the spacecraft attitude, is typically required with high accuracy and reliability, often under varying dynamic conditions. Attitude determination is therefore a central component of spacecraft guidance, navigation, and control systems.

Star trackers are among the most accurate attitude sensors available and determine spacecraft orientation by observing stars and matching measured star positions to an onboard catalog. At DTU Space, a high precision star tracker has been developed that is capable of achieving attitude accuracies on the order of arcseconds [1]. Such performance makes star trackers well suited for missions requiring precise absolute attitude knowledge. However, their accuracy and availability depend on image quality, and performance degrades when the spacecraft experiences rapid rotational motion. In high angular velocity scenarios, motion blur can significantly reduce centroiding accuracy or prevent successful star identification altogether [2].

To provide high rate attitude information and maintain observability during dynamic motion, star trackers are commonly complemented by gyroscopes. Gyroscopes measure angular velocity and allow attitude to be propagated between star tracker updates. Unlike star trackers, gyroscopes do not provide absolute attitude information, and their measurements must be integrated over time. This integration process is sensitive to error accumulation, particularly when the rotation axis varies with time. In such cases, coning motion arises due to the non-commutativity of finite rotations, introducing systematic errors into gyroscope attitude propagation if not properly compensated [3].

These limitations motivate the development and evaluation of robust attitude propagation and compensation techniques. This thesis addresses two related challenges. The first part investigates the degradation of star tracker images under rotational motion and explores both image denoising and deblurring techniques aimed at mitigating motion induced blur and improving centroiding performance. The second part focuses on coning error compensation for gyroscope attitude propagation, examining different kinematic integration methods and analyzing their accuracy and computational complexity.

Experimental data collected using the DTU star tracker are used to validate the investigated methods. Coning compensation techniques are evaluated using both simulation and experimental measurements, while image deblurring methods are applied to a representative star image.

This thesis is structured as follows. Section 2 introduces the mathematical representations of spacecraft attitude. Section 3 provides an overview of commonly used attitude sensors. Section 4 develops the image degradation model for star trackers under rotational motion and outlines the corresponding denoising and deblurring techniques. Section 5 presents the kinematic formulation of attitude propagation, introduces coning motion, and describes the considered integration methods.

Section 6 evaluates image restoration methods on a representative star field image. Subsequently, Section 7 assesses the kinematic integration methods through simulation and experimental validation using collected star tracker data.

Finally, Section 8 summarizes the main findings and discusses directions for future research. Additional experimental results and supplementary material are provided in the appendices.

2 Mathematical Representations of Spacecraft Attitude

2.1 Reference Frames

A *reference frame* is a coordinate system used to describe the position, velocity, and orientation of physical quantities. In rigid body dynamics, reference frames provide the basis relative to which vectors are expressed. Because of this, a single physical vector can have different coordinate representations depending on the frame in which it is expressed.

In this work, all kinematic quantities are expressed in right handed, orthonormal Cartesian reference frames. Two frames are used throughout:

- The *inertial frame*, denoted by the superscript n , which is taken to be fixed in space.
- The *body frame*, denoted by the superscript b , which is rigidly attached to the spacecraft and rotates with it.

A vector expressed in the body frame is written as $\mathbf{r}^b$, and the same physical vector expressed in the inertial frame is $\mathbf{r}^n$. The relationship between these representations is determined by the spacecraft's attitude, which may be described using any of the attitude parameterizations introduced in Section 2.2. Each of these representations encodes the same underlying rotation between the inertial and body frames. The distinction between *intrinsic* and *extrinsic* rotations in Section 2.1.2 is always understood relative to these frame conventions.

2.1.1 Important Reference Frames

Several reference frames are commonly used in spacecraft dynamics and navigation. The *Earth-Centered Earth-Fixed* (ECEF) frame is widely used for Earth related applications. Its origin is located at the center of the Earth, and its axes rotate with the Earth, making it well suited for describing the motion of objects relative to the Earth's surface.

In contrast, inertial reference frames do not rotate with the Earth and are therefore preferred for attitude analysis. A commonly used inertial frame is the *Earth-Centered Inertial* (ECI) frame, which also has its origin at the Earth's center but remains fixed with respect to the distant stars. Although the ECI frame experiences a small linear acceleration due to the Earth's motion about the Sun, this effect is negligible for attitude determination problems.

At a more fundamental level, the *International Celestial Reference Frame* (ICRF) defines an inertial coordinate system whose axes are fixed relative to the positions of distant celestial objects, with its origin at the center of mass of the solar system, the z -axis aligned with the Earth's mean north pole, and the x -axis aligned with the vernal equinox [4].

2.1.2 Extrinsic vs. Intrinsic Rotations

Extrinsic rotations are applied with respect to inertial reference axes, whereas intrinsic rotations are applied with respect to the body axes after they have been rotated. In the following, a simple example is presented to demonstrate an intuitive explanation of the two, rather than a rigorous mathematical proof.

Consider a two step rotation composed of a rotation about the body's x -axis followed by a rotation about the body's y -axis. In the intrinsic case, this sequence is applied as

$$\mathbf{v}'_b = R_{1b}(\alpha) \mathbf{v}_b, \quad (2.1)$$

$$\mathbf{v}''_b = R'_{2b}(\beta) \mathbf{v}'_b, \quad (2.2)$$

$$R_{\text{intr}} = R'_{2b}(\beta) R_{1b}(\alpha), \quad (2.3)$$

where $R_{1b}(\alpha)$ is the rotation about the initial body x -axis, and $R'_{2b}(\beta)$ is the rotation about the body y -axis *after* the first rotation. Primes indicate axes expressed in the updated (already rotated) body frame.

The same physical motion can be represented using extrinsic rotations, in which all rotations are described with respect to a fixed inertial frame:

$$\mathbf{v}'_n = R_{1n}(\alpha) \mathbf{v}_n, \quad (2.4)$$

$$\mathbf{v}''_n = R_{2n}(\beta) \mathbf{v}'_n, \quad (2.5)$$

$$R_{\text{ext}} = R_{2n}(\beta) R_{1n}(\alpha). \quad (2.6)$$

Here, $R_{1n}(\alpha)$ and $R_{2n}(\beta)$ denote rotations about the body x - and y -axes, respectively, but expressed in the inertial coordinate frame.

Initially the body and inertial axes are aligned. The extrinsic rotation can be written in terms of the intrinsic rotations by reversing their order:

$$R_{\text{ext}} = R_{2n}(\beta) R_{1n}(\alpha) = R_{1b}(\alpha) R'_{2b}(\beta), \quad (2.7)$$

because

$$R_{1n}(\alpha) = R_{1b}(\alpha), \quad (2.8)$$

$$R'_{2b}(\beta) = R_{1n}^T(\alpha) R_{2n}(\beta) R_{1n}(\alpha). \quad (2.9)$$

This reasoning generalizes to any number of sequential rotations.

In spacecraft attitude determination, this distinction matters because gyroscopes measure incremental rotations in the body frame, yielding an intrinsic sequence. To obtain the spacecraft's attitude in an inertial frame, these intrinsic rotations must be combined into a single extrinsic rotation, by chaining them in the reversed order.

2.2 Attitude Representations

Attitude representation is a way to describe a rotation from one reference frame to another. In general, this rotation is the rotation from a fixed reference frame in space to the body's reference frame. For spacecraft, it typically describes the orientation of the body frame with respect to a reference frame described in Section 2.1.1.

Euler's rotation theorem states a fundamental theorem in attitude determination: any orientation can be expressed as a single rotation about a fixed axis. This also means that any number of successive rotations can be simplified to a single axis and angle of rotation.

Different mathematical representations can be used to describe the attitude of a body with respect to a reference coordinate system. Each representation involves trade-offs with respect to interpretability, parameterization, and computational efficiency. In the following subsections, a number of them are described.

2.2.1 Direction Cosine Matrix

Direction Cosine Matrix (*DCM*), $\mathbf{C}_b^n$, is a 3×3 matrix, where each column represents the body's basis vectors expressed in the reference frame. Its component form is given with (2.10), where an element in the i th row and j th column represents the cosine of the angle between the i -axis of the reference frame and j -axis of the body frame.

$$\mathbf{C}_b^n = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix} \quad (2.10)$$

To express a vector, $\mathbf{r}^b$, defined in body axes, as a vector $\mathbf{r}^n$ defined in inertial axes, the following equation is used:

$$\mathbf{r}^n = \mathbf{C}_b^n \mathbf{r}^b \quad (2.11)$$

The DCM offers an intuitive geometric interpretation and can be directly used for vector transformations. Its main disadvantages are the need to store nine parameters for a quantity of three degrees of freedom and the requirement for periodic re-orthogonalization to prevent numerical drift [5].

2.2.2 Rotation Vector

The Euler axis ($\mathbf{e}$) and angle (ϕ) of rotation can be combined into a single 3D Rotation Vector (ϕ), expressed as:

$$\phi = \phi \mathbf{e} \quad (2.12)$$

This gives a very compact, three parameter representation of a rotation. In addition, strapdown sensors such as MEMS gyroscopes measure incremental rotations that can be naturally interpreted as rotation vectors, as explained in Section 5.1. The main drawbacks are that the rotation vector cannot be used directly for frame transformations without conversion to a DCM or quaternion, and that the parameterization becomes singular as the rotation angle approaches π . Further details can be found in [4].

2.2.3 Euler Angles

As already stated, successive rotations can be combined into a single rotation. Using the same principle, a single rotation can be decomposed into multiple successive rotations. Euler angles define three angles around three axes which when applied sequentially compose the full rotation.

The rotation axes are taken from the set of basis vectors. It is important to note that two successive rotations (using Euler angles) must be around two different axes. With this constraint in mind, a total of 12 different Euler angles combinations are possible, as presented in [4]. A convention is to label the first angle ψ , second θ and third ϕ .

In aerospace, a common set of Euler angles is Z-Y-X¹:

¹An aircraft first yaws on the runway, then pitches when taking off, and finally rolls in flight

- First, a *yaw* rotation of angle ψ is applied about the reference Z-axis
- Second, a *pitch* rotation of angle θ is applied about the new Y-axis
- Third, a *roll* rotation of angle ϕ is applied about the new X-axis

These rotations are intrinsic and are therefore applied about the successively rotated body axes. In the equivalent extrinsic formulation, the same overall rotation is obtained by applying the rotations in reverse order with respect to the fixed reference axes, as described in (2.7), for example when forming a single DCM. The total combined extrinsic rotation and each individual rotation, $R_z(\psi)$, $R_y(\theta)$ and $R_x(\phi)$, are given below:

$$\mathbf{C}_b^n = R_z(\psi) R_y(\theta) R_x(\phi) = \underbrace{\begin{bmatrix} c\psi & -s\psi & 0 \\ s\psi & c\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{R_z(\psi)} \underbrace{\begin{bmatrix} c\theta & 0 & s\theta \\ 0 & 1 & 0 \\ -s\theta & 0 & c\theta \end{bmatrix}}_{R_y(\theta)} \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & c\phi & -s\phi \\ 0 & s\phi & c\phi \end{bmatrix}}_{R_x(\phi)} \quad (2.13)$$

$$= \begin{bmatrix} c\psi c\theta & s\phi s\theta c\psi - s\psi c\phi & s\theta c\phi c\psi + s\phi s\psi \\ s\psi c\theta & s\phi s\theta s\psi + c\phi c\psi & s\psi s\theta c\phi - s\phi c\psi \\ -s\theta & s\phi c\theta & c\phi c\theta \end{bmatrix}. \quad (2.14)$$

With $c_\psi = \cos \psi$, $s_\psi = \sin \psi$ and analogous for θ and ϕ .

The main advantage of Euler angles is their intuitive physical interpretation, which makes them particularly suitable for visualization and qualitative analysis. They provide a minimal three parameter description of orientation and can be easily converted into a DCM. However, all Euler angle parametrizations are susceptible to gimbal lock, resulting in a loss of one degree of freedom at specific configurations.

Gimbal Lock

At certain axis alignments, Euler-angle representations lose a degree of freedom, which is known as *gimbal lock*. In the Z–Y–X representation, when $\theta = \pm\pi/2$, (2.14) becomes

$$\mathbf{C}_b^n = \begin{bmatrix} 0 & \pm \sin(\phi \mp \psi) & \pm \cos(\phi \mp \psi) \\ 0 & \cos(\phi \mp \psi) & -\sin(\phi \mp \psi) \\ \mp 1 & 0 & 0 \end{bmatrix}. \quad (2.15)$$

In (2.15), only the sum or difference of $\psi \pm \phi$ is calculated, meaning that a change in yaw would be equivalent to a roll change of the same amount and vice versa.

2.2.4 Quaternions

A quaternion is a mathematical structure that can be defined by the following equation:

$$\mathbf{q}(\mathbf{e}, \phi) = \begin{bmatrix} \mathbf{e} \sin(\phi/2) \\ \cos(\phi/2) \end{bmatrix} = \begin{bmatrix} \mathbf{q}_{1:3} \\ q_4 \end{bmatrix} \quad (2.16)$$

where $\mathbf{e}$ and ϕ are the parameters of the rotation vector from (2.12). $\mathbf{q}_{1:3} = [q_x \ q_y \ q_z]^T$ is the imaginary/vector part of $\mathbf{q}$, and q_4 is the scalar part.

To represent a rotation, the quaternion needs to be a *unit quaternion*, i.e. it must satisfy $\|\mathbf{q}\| = 1$.

The Hamilton product between two quaternions, $\odot$, is defined as follows:

$$\mathbf{q} \odot \mathbf{p} = \begin{bmatrix} \mathbf{q}_{1:3} \\ q_4 \end{bmatrix} \odot \begin{bmatrix} \mathbf{p}_{1:3} \\ p_4 \end{bmatrix} = \begin{bmatrix} q_4 \mathbf{p}_{1:3} + p_4 \mathbf{q}_{1:3} + \mathbf{q}_{1:3} \times \mathbf{p}_{1:3} \\ q_4 p_4 - \mathbf{q}_{1:3} \cdot \mathbf{p}_{1:3} \end{bmatrix} \quad (2.17)$$

$\mathbf{q}^*$ is the *conjugate* of $\mathbf{q}$ defined as:

$$\mathbf{q}^* = \begin{bmatrix} -\mathbf{q}_{1:3} \\ q_4 \end{bmatrix} \quad (2.18)$$

where the following property holds:

$$(\mathbf{q} \odot \mathbf{p})^* = \mathbf{p}^* \odot \mathbf{q}^* \quad (2.19)$$

A 3D vector, $\mathbf{v} = [v_x \ v_y \ v_z]^T$, can be embedded into a quaternion the following way:

$$\mathbf{q}_v = \begin{bmatrix} \mathbf{v} \\ 0 \end{bmatrix} = \begin{bmatrix} v_x \\ v_y \\ v_z \\ 0 \end{bmatrix} \quad (2.20)$$

To rotate a 3D vector, $\mathbf{v}$, by a quaternion $\mathbf{q}$, it first needs to be embedded as a quaternion $\mathbf{q}_v$ by (2.20). Then the following formula is used:

$$\mathbf{q}'_v = \mathbf{q} \odot \mathbf{q}_v \odot \mathbf{q}^* \quad (2.21)$$

The vector part $\mathbf{v}'$ is then extracted from $\mathbf{q}'_v$ which is the rotated 3D vector.

Under the assumption that individual quaternions represent intrinsic rotations, multiple rotations (where $\mathbf{q}_1$ occurred first, $\mathbf{q}_2$ second) can be chained together and applied to a 3D vector:

$$\mathbf{q}_{total} = \mathbf{q}_1 \odot \mathbf{q}_2 \quad (2.22)$$

$$\mathbf{q}'_v = \mathbf{q}_{total} \odot \mathbf{q}_v \odot \mathbf{q}_{total}^* = \mathbf{q}_1 \odot \mathbf{q}_2 \odot \mathbf{q}_v \odot \mathbf{q}_2^* \odot \mathbf{q}_1^* \quad (2.23)$$

Due to their computational efficiency and immunity to gimbal lock, quaternions are widely used for attitude representation [6] and serve as the primary representation in this work. Their main disadvantage lies in their limited physical interpretability compared to more intuitive parametrizations such as Euler angles.

A more in depth description of quaternions and their properties can be found in [4, 6, 7, 8].

Error Quaternions

To calculate the error, $\delta \mathbf{q}$, between two quaternions, the following definition is used:

$$\delta \mathbf{q} \equiv \begin{bmatrix} \delta \mathbf{q}_{1:3} \\ \delta q_4 \end{bmatrix} = \bar{\mathbf{q}}^* \odot \mathbf{q} \quad (2.24)$$

where $\bar{\mathbf{q}}$ is the estimated quaternion and $\mathbf{q}$ is the desired quaternion. $\delta \mathbf{q}$ can be converted into a rotation angle as described in equation (A.2) which is suitable for visualizations and comparisons.

3 Attitude Sensors

The attitude of the spacecraft can be determined using sensors that refer to either external physical phenomena or internal inertial measurements. External reference sensors infer attitude by observing known environmental features, such as stars, the Sun, the Earth's horizon, or the geomagnetic field. Inertial sensors are based solely on internal measurements and provide attitude information through integration over time.

Common attitude sensors include gyroscopes that measure angular velocity, star trackers that determine absolute attitude from stellar observations, Sun sensors based on observed Sun's location, horizon sensors for Earth pointing spacecraft, and magnetometers which require an accurate magnetic field model [4, 5]. This work focuses on gyroscopes and star trackers, as they are the primary sensors of the attitude determination system considered.

3.1 Gyroscopes

Gyroscopes measure angular velocity with respect to inertial space and are a fundamental component of inertial navigation systems. They are often used together with accelerometers in an inertial measurement unit (IMU) and provide high rate rotational information for attitude propagation.

Three main classes of gyroscopes are used in spacecraft applications: spinning mass gyroscopes, optical gyroscopes, and Coriolis vibratory gyroscopes (CVGs) [4, 5]. Spinning mass gyroscopes operate on the principle of conservation of angular momentum, where a rapidly rotating mass tends to maintain its axis of rotation in inertial space. Optical gyroscopes exploit the Sagnac effect by measuring phase differences between light beams propagating in opposite directions in a closed rotating path. CVGs, including MEMS gyroscopes, rely on the Coriolis force acting on a vibrating structure to sense angular motion. Today, CVGs are widely used due to their favorable balance between small size, low power consumption, low cost, and adequate accuracy.

Modern spacecraft commonly employ strapdown gyroscopes, which are rigidly attached to the spacecraft body. Compared to earlier spinning mass gyroscopes, strapdown systems are smaller, more robust, and less mechanically complex but require increased computational effort.

In star tracker systems, strapdown gyroscopes are commonly used to assist the star tracking process by providing high rate estimates of the spacecraft's rotational motion, a role that is described in more detail in Section 3.2.1.

3.1.1 MEMS Gyroscopes

Micro-electro-mechanical systems (MEMS) gyroscopes are a class of Coriolis vibratory gyroscopes. An oscillating mass experiences Coriolis acceleration when subjected to rotation, inducing a secondary vibration that is proportional to the angular velocity and can be measured electronically as a change in capacitance.

A commonly used continuous time measurement model for a three axis rate gyroscope is

$$\boldsymbol{\omega}(t) = \boldsymbol{\omega}_{\text{true}}(t) + \boldsymbol{\beta}_{\text{true}}(t) + \boldsymbol{\eta}_v(t), \quad (3.1)$$

$$\dot{\boldsymbol{\beta}}_{\text{true}}(t) = \boldsymbol{\eta}_u(t), \quad (3.2)$$

where ω is the measured angular rate, ω_{true} is the true angular rate, β_{true} is the gyro bias, and $\eta_v(t)$ and $\eta_u(t)$ are independent zero mean Gaussian white noise processes with standard deviations σ_v and σ_u , respectively [4].

Including scale factors and axis misalignment yields the more general model

$$\omega = (\mathbf{I}_3 + \mathbf{S}_{\text{true}}) \omega_{\text{true}} + \beta_{\text{true}} + \nu_v, \quad (3.3)$$

where $\mathbf{S}_{\text{true}}$ contains scale factor and misalignment errors.

Error Sources

The dominant error sources in MEMS gyroscopes follow directly from the above models. Scale factor and axis misalignment errors distort the measured angular rate through the matrix $\mathbf{S}_{\text{true}}$. Bias errors include a fixed bias, a turn on bias that varies between power cycles, and a time varying bias commonly modeled as a random walk. Bias errors must be compensated for as they otherwise accumulate through integration and lead to significant attitude drift.

3.2 Star Trackers

A star tracker is an optical attitude sensor that determines spacecraft orientation by observing stars within its field of view. It essentially consists of a camera with slightly defocused optics, causing the light from each star to be distributed over multiple pixels. This enables subpixel centroid estimation of star locations with high accuracy [1, 4].

By comparing the measured star centroids with known star positions from an onboard catalog, the spacecraft attitude can be determined. Modern star trackers achieve sub-arcsecond accuracy, making them the most precise attitude determination sensors currently available for spacecraft.

3.2.1 Modes of Operation

Star trackers typically operate in two modes, depending on whether prior attitude information is available¹.

Initial Attitude Acquisition

Initial attitude acquisition, often referred to as the *lost-in-space* problem, is performed when no prior knowledge of the spacecraft attitude is available. This mode is computationally intensive and relies solely on the observed star field.

A common acquisition algorithm proceeds as follows [9, 10]:

1. A candidate star triangle is formed by measuring the angular separations between three observed stars and comparing them with entries in the star catalog, as shown in Figure 3.1. If the triangle is unique, it is accepted; otherwise, a new triangle is selected and tested.
2. A reference star is then searched for by replacing one vertex of the triangle with another observed star and checking whether the resulting triangle is unique. If no such star is found, a new candidate triangle is formed and the process returns to step 1. If successful, the four stars are considered identified.

¹Due to computational limitations, earlier generations of star trackers required prior attitude information and were therefore commonly combined with other attitude sensors to provide initial estimates.

3. The remaining observed stars are iteratively tested for consistent matches with catalog entries using the same technique as in step 2, and all valid correspondences are retained.
4. Once a sufficient number of star correspondences has been established, the spacecraft attitude is determined by computing the rotation that best aligns the measured star unit vectors with their inertial frame counterparts [4, 11].

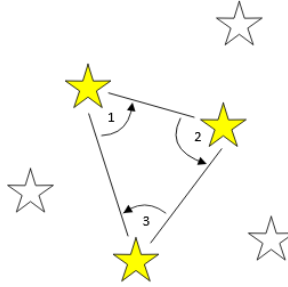


Figure 3.1: Star triangle used in the acquisition algorithm

Tracking Mode

In tracking mode, the star tracker continuously follows a set of previously identified stars. Using the prior attitude estimate and an estimate of the spacecraft's rotational motion obtained from the gyroscope, regions of interest (ROIs) are predicted for each star in the next image, as seen in Figure 3.2. If a star is lost from the field of view, the tracker finds another star, which should ideally be well separated from the rest of the tracked stars. The brightest pixels within each ROI are identified, star centroids are recomputed, and the attitude is updated. This mode is computationally efficient and provides high accuracy and higher rate attitude updates.



Figure 3.2: ROIs in a star tracker image.

4 Star Tracker Image Modeling and Deblurring

In the presence of rotational motion, star tracker images are smeared across the image plane, causing stars to appear as short trajectories rather than point sources. This motion blur reduces the signal-to-noise ratio (SNR) of the image, degrades centroiding accuracy, and can ultimately prevent reliable star detection. This chapter explores image denoising and motion deblurring approaches aimed at recovering usable star information under dynamic conditions. In general, the motion underlying image blur can be estimated either directly from the image content alone [12] or with the aid of gyroscope measurements [2, 13, 14]. In this work, the focus is placed exclusively on the latter approach, where gyroscope measurements are used to inform the blur model.

4.1 Star Image Formation

Figure 4.1 illustrates the two operating conditions of a star tracker: static and dynamic. Under static conditions, each star is imaged as a compact two dimensional Gaussian intensity distribution. During rotational motion, this distribution is smeared along the direction of motion, forming an elongated trajectory over the exposure interval. As the rotational speed increases, the energy is spread over a larger number of pixels, reducing the effective SNR further [14].

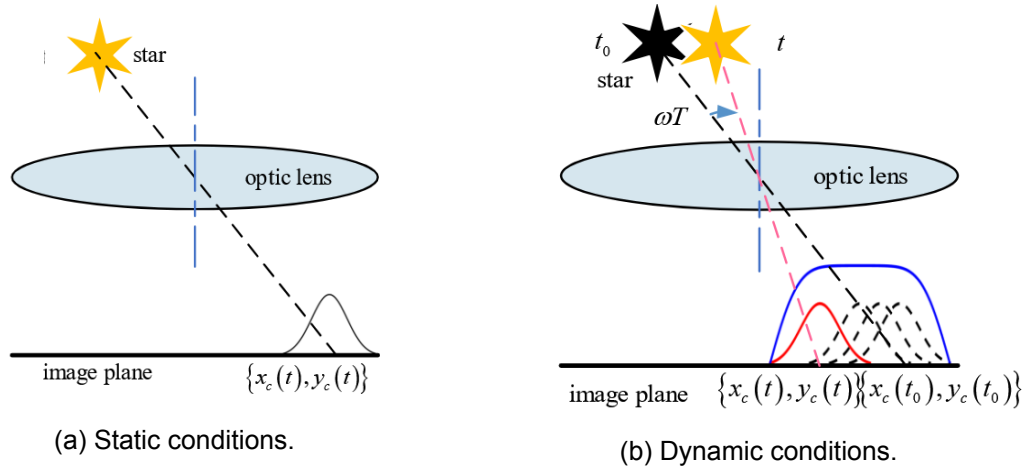


Figure 4.1: Star tracker image formation under static and dynamic conditions (reproduced from [14]).

For the following analysis, the camera coordinate frame is assumed to be aligned with the gyroscope axes, with the x -axis along the image width, the y -axis along the image height, and the z -axis pointing from the image plane toward the optical center. The camera frame origin is located at the optical center, while image coordinates are defined with their origin at the center of the image plane. This convention is schematically illustrated in Figure 4.2.

¹For visual clarity, the coordinate axes are shown offset from the optical center. Under a strict pinhole camera model, the image would also appear inverted. This inversion is omitted here without loss of generality.

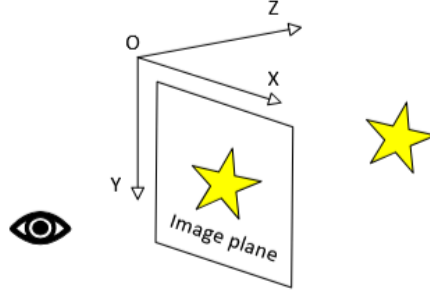


Figure 4.2: Camera coordinate system used in the analysis.¹

4.2 Image Degradation Model

Two main sources of image degradation are motion induced smearing of stars and sensor noise. Both effects reduce the effective SNR in the image and limit the accuracy of star detection and centroid estimation.

4.2.1 Motion Blur

The objective of the motion blur model is to describe the trajectory of star centroids on the image plane during the exposure time. This trajectory defines the blur kernel used by the deblurring algorithms. Throughout the following analysis, it is assumed that the rotational motion passes through the optical center of the camera.

Two modeling approaches are commonly found in the literature. The first is a *coupled* model, where the image motion is directly expressed as a function of the angular velocity components about all three axes [14]. The second is a *decoupled* model, in which the motion is decomposed into rotation about the optical axis and rotation about the transverse axes, and the resulting blurs are applied sequentially [2].

4.2.2 Coupled Motion Model

The coupled model provides a first order approximation of the star motion on the image plane when rotations about all three axes are present. Although still an approximation, it more closely reflects the true camera motion than the decoupled model.

Following [2], the motion of a point on the image plane over a short time interval Δt can be written as

$$\begin{cases} x(t_0 + \Delta t) = x(t_0) + y(t_0)\omega_z\Delta t + f\omega_y\Delta t, \\ y(t_0 + \Delta t) = y(t_0) - x(t_0)\omega_z\Delta t - f\omega_x\Delta t, \end{cases} \quad (4.1)$$

where x and y denote image coordinates, f is the focal length expressed in pixels, and ω_x , ω_y , and ω_z are the angular velocity components about the camera axes.

Because the image motion depends explicitly on the initial pixel location through the terms $x(t_0)\omega_z\Delta t$ and $y(t_0)\omega_z\Delta t$, each pixel follows a different trajectory. As a result, the blur kernel is generally spatially variant across the image.

4.2.3 Decoupled Motion Model

The decoupled model separates the rotational motion into two components and is a good approximation when either the rotation about the optical axis is small or the rotations about the transverse axes are small.

Rotation About the Optical Axis

If rotation occurs only about the optical axis, setting $\omega_x = \omega_y = 0$ in (4.1) yields

$$\begin{cases} x(t_0 + \Delta t) = x(t_0) + y(t_0) \omega_z \Delta t, \\ y(t_0 + \Delta t) = y(t_0) - x(t_0) \omega_z \Delta t. \end{cases} \quad (4.2)$$

As in the coupled case, the motion depends on the pixel location, and the blur kernel is therefore spatially variant.

Rotation About the Transverse Axes

If rotation occurs only about the x and y axes, setting $\omega_z = 0$ yields

$$\begin{cases} x(t_0 + \Delta t) = x(t_0) + f \omega_y \Delta t, \\ y(t_0 + \Delta t) = y(t_0) - f \omega_x \Delta t. \end{cases} \quad (4.3)$$

In this case, the motion is independent of the pixel location. The terms $x(t_0)$ and $y(t_0)$ represent only a constant offset and do not influence the shape of the blur kernel. Consequently, a single, spatially invariant blur kernel applies to the entire image.

Combination of Rotational Components

When rotation about all three axes is present, the decoupled model approximates the total motion blur by sequentially applying the transverse axis blur followed by the optical axis blur. In the context of image restoration, the inverse order must be used, such that deblurring is performed by first compensating for the optical axis rotation and then for the transverse axis rotation [15].

4.2.4 Difference Between the Coupled and Decoupled Models

A difference between the coupled and decoupled models arises when both rotation about the optical axis and the transverse axes is present. Figure 4.3 illustrates the qualitative difference between the two models. The left image shows the original sharp star image, while the middle and right images show the blurred results obtained using the coupled and decoupled models, respectively.

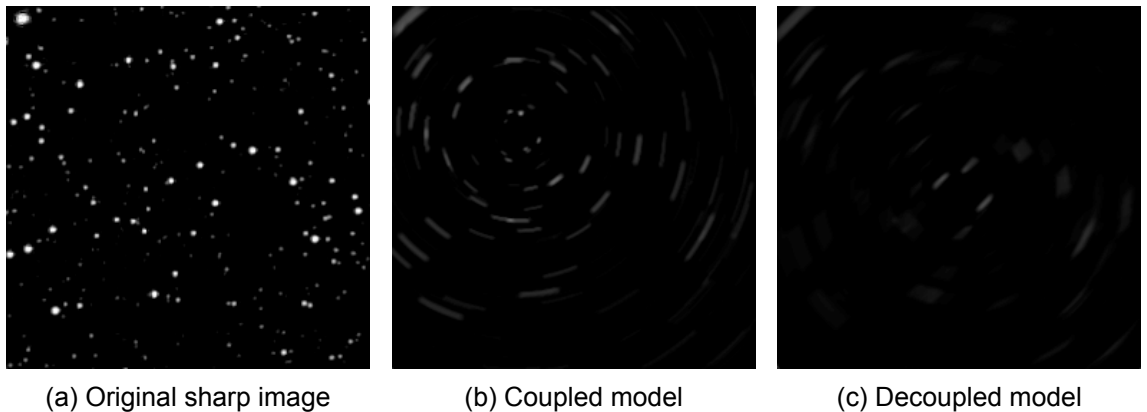


Figure 4.3: Difference between coupled and decoupled blur models for $\omega_x \neq 0$, $\omega_y \neq 0$, and $\omega_z \neq 0$. Image credit: NASA/JPL [16]

In the coupled model, the effective center of rotation in the image plane shifts away from the principal point. For angular velocities $(\omega_x, \omega_y, \omega_z)$, focal length f , and principal point

(c_x, c_y) expressed in pixel coordinates, the rotation center (u_c, v_c) in pixel coordinates is given by

$$u_c = c_x - \frac{f \omega_x}{\omega_z}, \quad (4.4)$$

$$v_c = c_y - \frac{f \omega_y}{\omega_z}. \quad (4.5)$$

In contrast, the decoupled model assumes that the rotation around the z -axis is centered at the principal point, so the image center remains the center of rotation. This difference leads to distinct blur patterns when all three angular velocity components are present.

4.2.5 Noise Model

Star tracker images are affected by additive noise, commonly modeled as a combination of Poisson noise, Gaussian white noise, and salt and pepper noise [2]. In this work, only zero mean Gaussian white noise and salt and pepper noise are considered. These noise sources reduce the effective SNR and further degrade image quality.

4.3 Image Denoising

Noise estimation and removal is the first step in image reconstruction, as deblurring algorithms are highly sensitive to noise and tend to amplify it during restoration. In this work, the denoising method proposed in [2] is adopted. The method is an improved Gaussian curvature filter that evaluates the consistency of a pixel with its local neighborhood by measuring its distance to multiple tangent planes defined over a 3×3 window.

The algorithm consists of three main steps.

4.3.1 Estimation of the Filtered Pixel Value

An estimated pixel value $\hat{f}(x, y)$ is first computed using a Gaussian curvature filter. As illustrated in Figure 4.4, eight tangent planes are constructed from different combinations of neighboring pixels within a 3×3 window. For each plane, the difference between the pixel value $f(x, y)$ and the average value of the corresponding tangent plane is calculated.

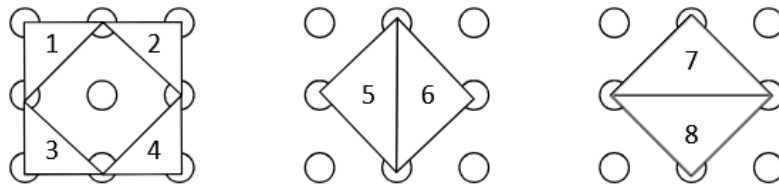


Figure 4.4: Tangent planes used by the Gaussian curvature filter. The eight planes are formed from different combinations of neighboring pixels in a 3×3 window. In the figure, the pixels contributing to each tangent plane are located beneath the vertices of the corresponding triangle.

For example, for the first tangent plane in Figure 4.4, the distance is given by

$$d_1 = \frac{f(x-1, y-1) + f(x-1, y) + f(x, y-1)}{3} - f(x, y). \quad (4.6)$$

The corresponding computation is performed for all eight tangent planes, yielding distances $\{d_n\}_{n=1}^8$. The estimated pixel value is then obtained by selecting the distance with

the smallest absolute value,

$$\hat{f}(x, y) = f(x, y) + d_m, \quad |d_m| = \min_n |d_n|. \quad (4.7)$$

4.3.2 Pixel Energy Computation

To distinguish noise contaminated pixels from valid signal pixels, an energy function is defined for each pixel,

$$E(x, y) = E_s(x, y) + E_d(x, y), \quad (4.8)$$

where E_s represents the potential energy required to adjust the pixel value to $\hat{f}(x, y)$,

$$E_s(x, y) = |f(x, y) - \hat{f}(x, y)|, \quad (4.9)$$

and E_d measures neighborhood inconsistency. The latter is computed as the sum of the four smallest values of a potential function evaluated between the central pixel and its neighbors,

$$E_d(x, y) = \sum_{k=1}^4 r_k, \quad (4.10)$$

$$A(x, y) = \phi(f(x, y) - f(k, l)), \quad (k, l) \in v(x, y), \quad (4.11)$$

$$\phi(t) = |t|^\alpha, \quad 1 \leq \alpha \leq 2, \quad (4.12)$$

where r_k denotes the k -th smallest value of $A(x, y)$ in the neighborhood $v(x, y)$. Noise contaminated pixels typically exhibit large intensity variations relative to their neighbors, resulting in high values of E_d .

4.3.3 Noise Detection and Pixel Update

Finally, the total energy $E(x, y)$ is compared against a predefined threshold. If $E(x, y)$ exceeds this threshold, the pixel is classified as noise and its value is replaced by $\hat{f}(x, y)$. Otherwise, the original pixel value is retained. In this way, only corrupted pixels are modified, allowing effective noise suppression while preserving the underlying star signal.

4.4 Image Deblurring Methods

After denoising, the next step in the image reconstruction process is deblurring. In this work, the denoised image is restored using blur kernels estimated from the motion models described in Section 4.2.

Two classical deconvolution approaches are considered: modified Wiener filter (referred to simply as Wiener filter in the following) and the Richardson–Lucy deconvolution algorithm. In all cases, the angular velocity ω is assumed to remain constant during the exposure interval, such that a single blur kernel can be used to model the motion over that period.

4.4.1 Wiener Deconvolution

Image degradation by motion blur can be modeled as a convolution between the latent sharp image and a blur kernel. By applying the discrete Fourier transform (DFT), convolution in the spatial domain becomes multiplication in the frequency domain, which makes deconvolution computationally efficient.

The Wiener filter provides a regularized solution to the deconvolution problem. The restored image in the frequency domain is given by

$$\hat{F}(u, v) = \frac{H^*(u, v)}{H(u, v)H^*(u, v) + K} G(u, v), \quad (4.13)$$

where $\hat{F}(u, v)$ denotes the Fourier transform of the restored image, $G(u, v)$ is the Fourier transform of the blurred image, $H(u, v)$ is the Fourier transform of the blur kernel, $H^*(u, v)$ is its complex conjugate, and K is an adjustable empirical parameter that balances noise suppression and image sharpness [13].

A prerequisite for applying the Wiener filter in this form is that the blur kernel is spatially invariant.

Decoupled Model

When the decoupled motion model described in Section 4.2.3 is assumed, the deblurring process is performed in two steps, as illustrated in Figure 4.5.

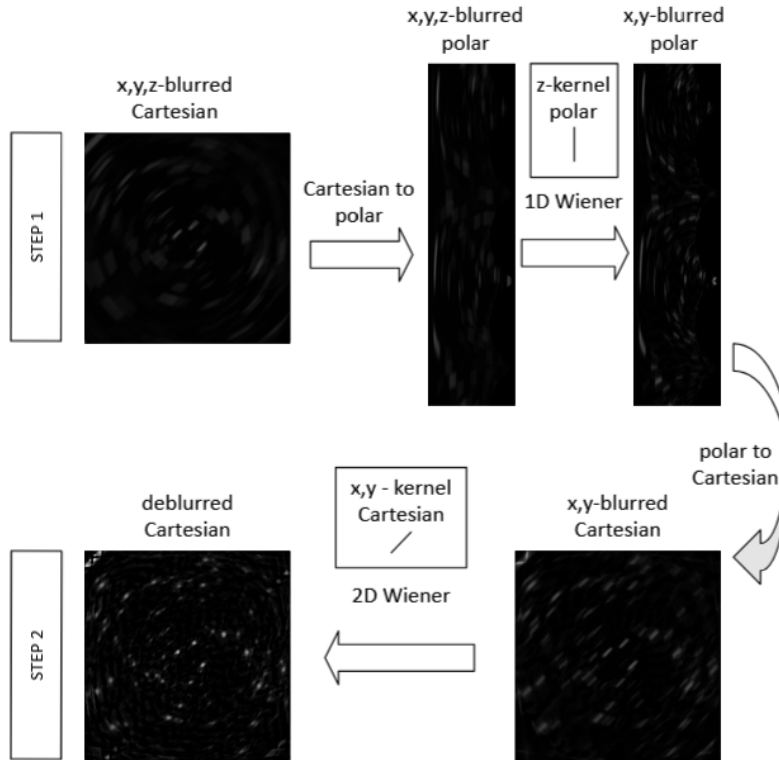


Figure 4.5: Two-step Wiener deblurring under the decoupled motion model.

1) Deconvolution in the z -direction For rotation about the z -axis, the blur kernel is spatially variant in Cartesian image coordinates (Section 4.2.3). To obtain a spatially invariant representation, the image is transformed into polar coordinates with the center of rotation at the image center.

In polar coordinates, rotation about the z -axis becomes a shift along the angular coordinate θ . A one dimensional Wiener filter, derived from (4.13), is applied along the θ direction. The blur kernel length in the polar image is given by

$$L_\theta = \frac{\theta_z}{2\pi} L_{\theta, \text{image}}, \quad (4.14)$$

where θ_z is the total angular displacement during exposure and $L_{\theta, \text{image}}$ is the total length of the θ dimension in the polar image.

After deconvolution, the image is transformed back to Cartesian coordinates.

2) Deconvolution in the x - y directions For rotations about the x and y axes, the blur kernel described in Section 4.2.3 is spatially invariant. Therefore, the two dimensional Wiener filter of (4.13) can be directly applied in Cartesian coordinates using the corresponding x - y kernel.

Coupled Model

When the coupled motion model described in Section 4.2.2 is assumed, the rotational center, (u_c, v_c) , must first be determined by (4.4) and (4.5).

If $\omega_z \neq 0$, the image is transformed into polar coordinates using (u_c, v_c) as the rotation center, and a one dimensional Wiener filter is applied in the angular direction.

If $\omega_z = 0$, no rotational center exists and the blur reduces to the x - y component only. In this case, the two dimensional Wiener filter (4.13) is applied directly in Cartesian coordinates.

4.4.2 Richardson–Lucy

The Richardson–Lucy (RL) algorithm is an iterative image restoration method based on Bayesian analysis [17]. It estimates the original image using a maximum likelihood approach. The general iterative form is given by [2]:

$$f^{(k+1)} = f^{(k)} \left[\left(\frac{g}{f^{(k)} * h} \right) \oplus h \right], \quad (4.15)$$

where $f^{(k)}$ is the image estimate at iteration k , g is the blurred image, h is the blur kernel, $*$ denotes convolution, and $\oplus$ denotes correlation.

If the blur kernel is spatially variant, a corresponding local kernel is used for each pixel.

Since (4.15) does not include a termination condition, the choice of the iteration number is critical. A small number of iterations results in incomplete restoration, while too many iterations increase computation time and amplify noise.

Automatic Termination Condition

To address the issue of the number of iterations, [2] proposes a termination criterion based on star centroid coordinates in three consecutive restored images:

$$\begin{cases} ||x^{k+1} - x^k| - |x^k - x^{k-1}|| \leq \varepsilon(\Delta x), \\ ||y^{k+1} - y^k| - |y^k - y^{k-1}|| \leq \varepsilon(\Delta y), \end{cases} \quad (4.16)$$

where (x^k, y^k) are the star centroid coordinates at iteration k , Δx and Δy are the estimated blur lengths in the respective directions based on the angular velocities, and ε is an empirically chosen scaling factor.

If multiple stars are present, the termination condition is considered satisfied when a pre-defined percentage of centroids fulfills (4.16).

Decoupled Model

In the decoupled model, the convolution with the blur kernels is performed sequentially. First, the spatially invariant x - y kernel (4.3) is applied, followed by the z -direction kernel (4.2).

When performing the correlation operation, the order is reversed: the z -kernel is applied first, followed by the x - y kernel, as described in 4.2.3.

Coupled Model

In the coupled model, the blur kernel derived from (4.1) is used directly as h in (4.15). The restoration then follows the standard RL update without kernel separation.

5 Attitude Kinematics

Unlike star trackers, gyroscopes measure the angular velocity of a spacecraft rather than its attitude directly. Consequently, a kinematic model is required to propagate the spacecraft attitude by integrating the measured angular velocity over time. The resulting attitude estimate is inherently relative and depends on the assumed initial condition.

5.1 Coning Motion

Bortz's Equation

The attitude kinematics of a rotating rigid body can be expressed in terms of the rotation vector ϕ , defined in (2.12), using Bortz's equation [18]. For angular velocity ω expressed in the body frame, the continuous time relationship is given by

$$\dot{\phi} = \omega + \frac{1}{2} \phi \times \omega + \frac{1}{\phi^2} \left(1 - \frac{\phi \sin \phi}{2(1 - \cos \phi)} \right) [\phi \times (\phi \times \omega)], \quad (5.1)$$

where $\phi = \|\phi\|$.

In gyroscope terminology, ω is the instantaneous angular velocity vector, while ϕ represents the finite rotation over a time interval. A rate gyroscope measures the angular velocity ω directly, whereas a rate integrating gyroscope outputs the accumulated rotation angle

$$\theta(t) = \int_{t_0}^t \omega(\tau) d\tau, \quad (5.2)$$

which in general differs from ϕ due to the non-commutativity of finite rotations [19].

Coning

Integrating Bortz's equation leads to the decomposition

$$\phi = \theta + \beta, \quad (5.3)$$

where θ is the integrated gyroscope output and β is the coning correction term. The coning term is given by

$$\beta = \int_{t_0}^t \left[\frac{1}{2} \phi \times \omega + \frac{1}{\phi^2} \left(1 - \frac{\phi \sin \phi}{2(1 - \cos \phi)} \right) \phi \times (\phi \times \omega) \right] d\tau. \quad (5.4)$$

Coning arises when the rotation vector itself changes direction during the integration interval, which commonly occurs in oscillatory or high frequency motions. This effect is a direct consequence of the non-commutativity of finite rotations. For rotation about a fixed axis, $\phi \times \omega = \mathbf{0}$ and the coning term vanishes [3].

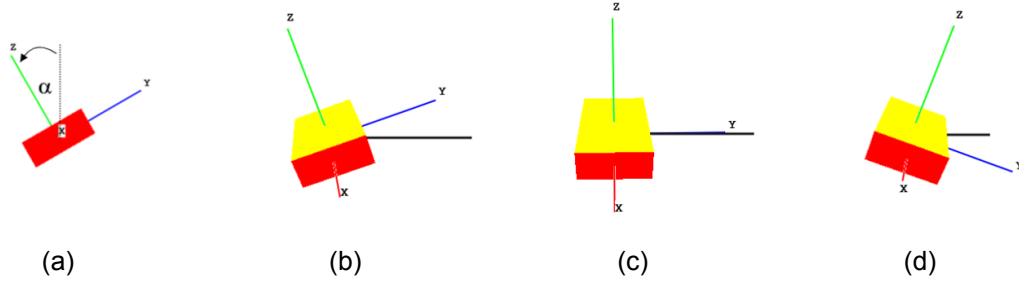


Figure 5.1: Illustration of pure coning motion. The instantaneous rotation axis is shown in black, while cone half-angle α is shown in the first panel.

Pure coning motion can be described by two parameters: the cone half-angle α and the coning frequency ω . The coning frequency ω characterizes how fast the instantaneous rotation axis itself precesses, while the half-angle α describes the magnitude of the body rotation about this axis. The term *coning* originates from the resulting geometry, where the body fixed z -axis traces out a cone of half-angle α about the inertial Z -axis. Under this motion, the coning correction term in (5.3) is maximized [20].

Figure 5.1 illustrates this motion. The instantaneous rotation axis is shown by the black axis, and the cone half-angle α is indicated in Fig. 5.1a. Figures 5.1b–d show the same coning motion at different phases of the coning cycle.

5.1.1 Coning Motion Representation

In the following, closed form expressions for pure coning motion are presented using both time dependent quaternions and angular velocity vectors [21]. The motion is defined such that the instantaneous rotation axis precesses about the body z -axis.

Quaternion Representation of Coning Motion

For coning motion with half-angle α and coning frequency ω , the quaternion describing the attitude as a function of time is given by

$$\mathbf{q}(t) = \begin{bmatrix} q_x(t) \\ q_y(t) \\ q_z(t) \\ q_w(t) \end{bmatrix} = \begin{bmatrix} \sin\left(\frac{\alpha}{2}\right) \cos(\omega t) \\ \sin\left(\frac{\alpha}{2}\right) \sin(\omega t) \\ 0 \\ \cos\left(\frac{\alpha}{2}\right) \end{bmatrix}. \quad (5.5)$$

This quaternion represents a rotation of constant magnitude α , whose axis traces a circular trajectory in the x - y plane about the z -axis.

Angular Velocity Representation

The corresponding body frame angular velocity that describes the coning motion in (5.5) is

$$\boldsymbol{\omega}(t) = \begin{bmatrix} -\omega \sin \alpha \sin(\omega t) \\ \omega \sin \alpha \cos(\omega t) \\ -2\omega \sin^2\left(\frac{\alpha}{2}\right) \end{bmatrix}. \quad (5.6)$$

The angular velocity has constant magnitude

$$\|\omega(t)\| = 2\omega \sin\left(\frac{\alpha}{2}\right), \quad (5.7)$$

while its direction varies continuously with time.

In a strapdown inertial system, an ideal rate gyroscope mounted in the body frame would directly measure the angular velocity vector $\omega(t)$ given in (5.6).

Approximations

For small rotation magnitudes, the nonlinear coefficient in (5.4) can be approximated using a Taylor series expansion [19]:

$$\frac{1}{\phi^2} \left(1 - \frac{\phi \sin \phi}{2(1 - \cos \phi)} \right) = \frac{1}{12} \left(1 + \frac{\phi^2}{60} + \dots \right). \quad (5.8)$$

Retaining only the terms up to third order results in the approximation

$$\beta \approx \int_{t_0}^t \left[\frac{1}{2} \phi \times \omega + \frac{1}{12} \phi \times (\phi \times \omega) \right] d\tau. \quad (5.9)$$

A further simplification is the Goodman–Robinson approximation, which replaces ϕ with the measured rotation vector θ and retains only first order terms:

$$\beta \approx \frac{1}{2} \int_{t_0}^t \theta \times \omega d\tau. \quad (5.10)$$

This approximation forms the basis of many classical coning compensation algorithms due to its simplicity and computational efficiency [19].

5.2 Attitude Propagation Methods

With the mathematical foundations of attitude kinematics established, this section presents practical methods for propagating attitude using gyroscope measurements. In particular, both single-speed and two-speed integration schemes are considered.

5.2.1 Two-Speed Algorithms

Two-speed algorithms are commonly used in inertial navigation systems with limited processing capability. The computational efficiency is improved by separating the high rate sensor sampling from the lower rate attitude update. Gyroscope measurements are acquired at the sensor sampling interval, while attitude updates are performed at a longer interval by combining multiple samples. This reduces the computational load associated with frequent attitude propagation, providing an effective compromise between accuracy and complexity [22, 23]. Figure 5.2 illustrates the two-speed update scheme, where each attitude update consists of five sensor updates.

Several multi sample coning algorithms have been proposed for two-speed implementations, including three sample algorithms such as those presented in [20, 21]. These three sample approaches are not considered further in this work, as the system used was implemented with a 20 sample sensor update interval.

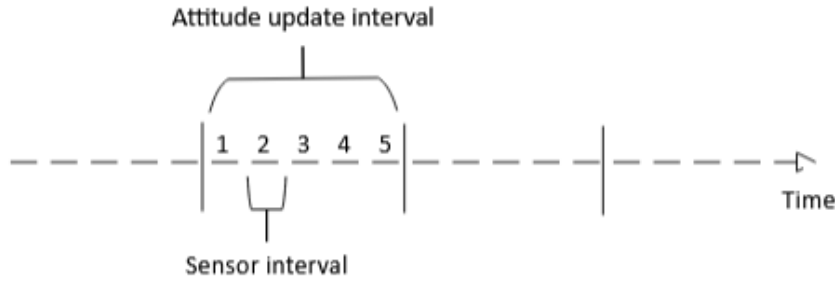


Figure 5.2: Two-speed update intervals. Gyroscope measurements are acquired at a high rate sensor sampling interval, while attitude updates are performed at a lower rate. In this example, each attitude update interval consists of five sensor sampling intervals.

Simple Kinematic Integration

The simplest approach to attitude propagation uses direct integration of the measured angular velocity without explicit coning compensation. This method is implemented in a two-speed framework, where gyroscope measurements are accumulated at the sensor sampling interval and the attitude is updated at a lower rate.

Simple kinematic integration neglects the coning term β of (5.3) and approximates the finite rotation only by the integrated angular velocity,

$$\phi \approx \theta. \quad (5.11)$$

Using this assumption, the rotation vector over one attitude update interval is obtained by summing the angular velocity measurements over the sensor sampling intervals. The discrete time rotation vector can then be written as

$$\theta = \sum_{i=1}^N \omega_i \Delta t, \quad (5.12)$$

where N is the number of gyroscope samples per attitude update and Δt is the sensor sampling period.

Once the rotation vector has been computed, it is converted into an incremental quaternion $\Delta \mathbf{q}$ using (2.16). The attitude is then propagated by chaining this incremental rotation with the previous attitude estimate using (2.22).

This integration scheme is computationally lightweight, as reflected in the operation counts reported in Table 5.1, but because it neglects the non-commutativity of finite rotations, it accumulates coning errors.

Rotation Vector Update

An improved attitude propagation method accounts for the coning effect by including a correction term in the rotation vector update. As with simple kinematic integration, this approach is typically implemented in a two-speed framework.

The coning term β of (5.3) is approximated in discrete time using the Goodman–Robinson formulation (5.10) together with the third order approximation (5.9). The finite rotation vector over one attitude update interval is then given by

$$\phi_m = \theta_m + \frac{1}{2} \sum_{k=1}^m \theta_k \times \Delta\theta_k + \frac{1}{12} \sum_{k=1}^m \Delta\theta_{k-1} \times \Delta\theta_k, \quad (5.13)$$

$$\theta_m = \sum_{k=1}^m \Delta\theta_k, \quad (5.14)$$

where $\Delta\theta_k = \omega_k \Delta t$ denotes the incremental rotation measured by the gyroscope at the k th sensor sample, m is the number of sensor samples per attitude update interval, and θ_k is the partial rotation vector accumulated up to sample k [19].

The resulting rotation vector is then used to update the attitude as described in Section 5.2.1.

Compared to simple kinematic integration, this method reduces coning error by explicitly accounting for the non-commutativity of successive rotations. However, residual errors remain due to the approximation of Bortz's equation and the fact that gyroscope measurements are available only at discrete time instants [20]. As summarized in Table 5.1, the rotation vector update method remains computationally lightweight and is significantly less expensive than higher order integration schemes or single speed algorithms.

5.2.2 Single-Speed Algorithm

When enough computing power is available, a single update interval is preferred, where the attitude update is performed at the same frequency as the sensor update, as it simplifies the implementation and reduces approximation errors.

Direct Quaternion Update

The direct quaternion update method propagates attitude using a single speed algorithm, in which the rotation vector over one sensor interval is computed directly, without accumulating intermediate rotation vectors over a longer update interval.

The rotation vector corresponding to one sensor interval is given by

$$\phi_k = \Delta\theta_k + \frac{1}{2} \Delta\theta_{k-1} \times \Delta\theta_k, \quad (5.15)$$

where $\Delta\theta_k = \omega_k \Delta t$ denotes the incremental rotation measured by the gyroscope at the k th sample [19].

The resulting rotation vector is then used to update the attitude as described in Section 5.2.1.

Because the update is performed at the sensor rate, this method avoids the accumulation of inter sample coning effects and is therefore more accurate than two-speed algorithms. The main drawback of the direct quaternion update is its increased computational cost, as quantified in Table 5.1, as quaternion updates and coning compensation must be performed at every sensor sample.

5.2.3 Computational Complexity

The computational cost of the attitude propagation algorithms is expressed in terms of the number of floating-point operations required per update cycle. Since different operations have different computational costs on typical embedded processors, the operation counts are reported by operation type rather than combined into a single scalar measure.

Table 5.1 summarizes the operation counts for each method. All values correspond to the same time window comprising 20 sensor samples. For the two-speed algorithms, this window consists of 20 sensor updates followed by a single attitude update. For the direct quaternion update, the same window corresponds to 20 attitude updates performed at the sensor sampling rate.

Table 5.1: Floating-point operation (FLOP) counts for 20 sensor samples.

Method	Add	Mul	Div	Sqrt	Trig
Simple kinematic integration	80	92	2	2	2
Rotation vector update	280	332	2	2	2
Direct quaternion update	520	880	40	40	40

6 Star Tracker Image Restoration Results

The image denoising and deblurring algorithms described in Sections 4.3 and 4.4 are evaluated in this section. The primary objective is to assess whether the proposed methods improve the SNR of degraded star tracker images and, consequently, enhance centroiding accuracy.

6.1 Methodology

All algorithms were tested on a publicly available star field image [16], from which a 256×256 pixel subregion was selected for analysis, shown in Figure 4.3a, in which 50 stars were identified. The original image was artificially degraded using the motion and noise models introduced in Section 4.2.

The exposure time was set to 0.25 s, the pixel size to $5 \mu\text{m}$, and the focal length to 20 mm.

All experiments were performed on a system equipped with a 13th Gen Intel(R) Core(TM) i5-13500H processor (2.60 GHz, 12 cores) and 32 GB RAM, running Windows 11. The algorithms were implemented and executed in Python 3.12.

Evaluation Metric

The performance of the restoration algorithms was evaluated using the average centroiding error.

Star centroids in each image were computed using the following procedure:

1. Thresholding the image (a threshold value of 100 was used).
2. Separating stars in the binary image using connected component labeling (connected positive pixels are treated as one star).
3. Discarding detections that were too small, too large, or too close to the image boundaries, in order to remove artifacts introduced by deblurring.
4. Computing the centroid of each remaining star using a weighted average of pixel intensities.

Centroids were calculated for both the original (non-degraded) image, used as ground truth, and the restored image. Each centroid in the restored image was matched to the nearest centroid in the ground truth image. The Euclidean distances between matched centroid pairs were then computed, and their average value was used as the final performance metric.

6.2 Results

6.2.1 Deblurring of Transverse Rotation

Figure 6.1 shows the effect of Wiener deconvolution and Richardson–Lucy (RL) deconvolution on an image degraded by transverse rotation with $\omega_x = \omega_y = 1 \text{ rad/s}$. No noise was added in this experiment.

The centroiding error of the blurred image was 2.8 pixels. After restoration, the error decreased to 0.29 pixels for the Wiener filter and 0.24 pixels for RL. Both methods restore subpixel centroiding accuracy, demonstrating a substantial improvement compared to the degraded image.

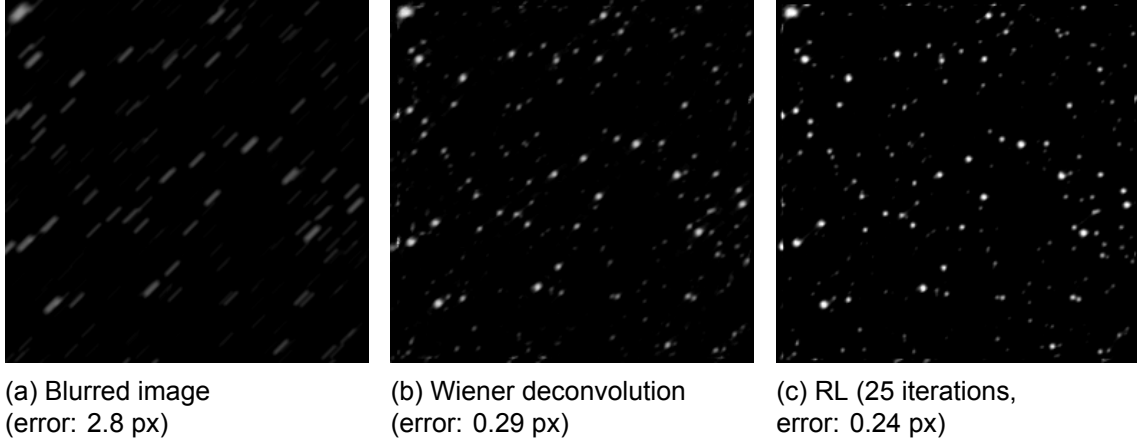


Figure 6.1: Deblurring results for transverse rotation with $\omega_x = \omega_y = 1$ rad/s.

6.2.2 RL Automatic Termination Condition

Figure 6.2 compares RL with a fixed number of 25 iterations and RL with the automatic termination condition described in Section 4.4.2. The image was degraded using coupled motion with $\omega_x = \omega_y = \omega_z = 1$ rad/s.

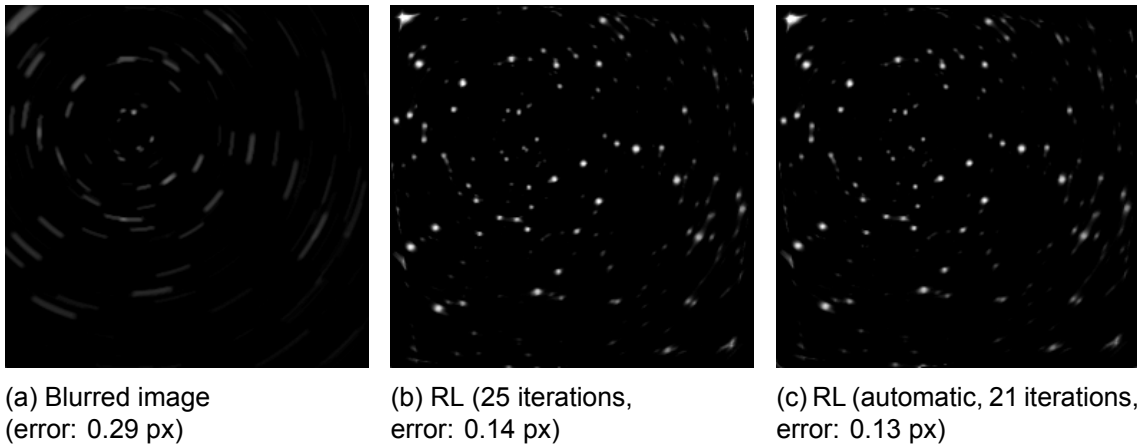


Figure 6.2: Comparison of RL with fixed and automatic termination.

The automatic termination condition achieved slightly lower error (0.13 pixels) using fewer iterations (21) compared to the fixed 25 iteration approach (0.14 pixels error). Both methods improve centroid accuracy relative to the blurred image (0.29 pixels error).

6.2.3 Denoising

Gaussian noise with standard deviation 13 (intensity units) and salt and pepper noise with probability 1% per pixel were added to the blurred image. Figure 6.3 shows the results before and after denoising, followed by Wiener deconvolution.

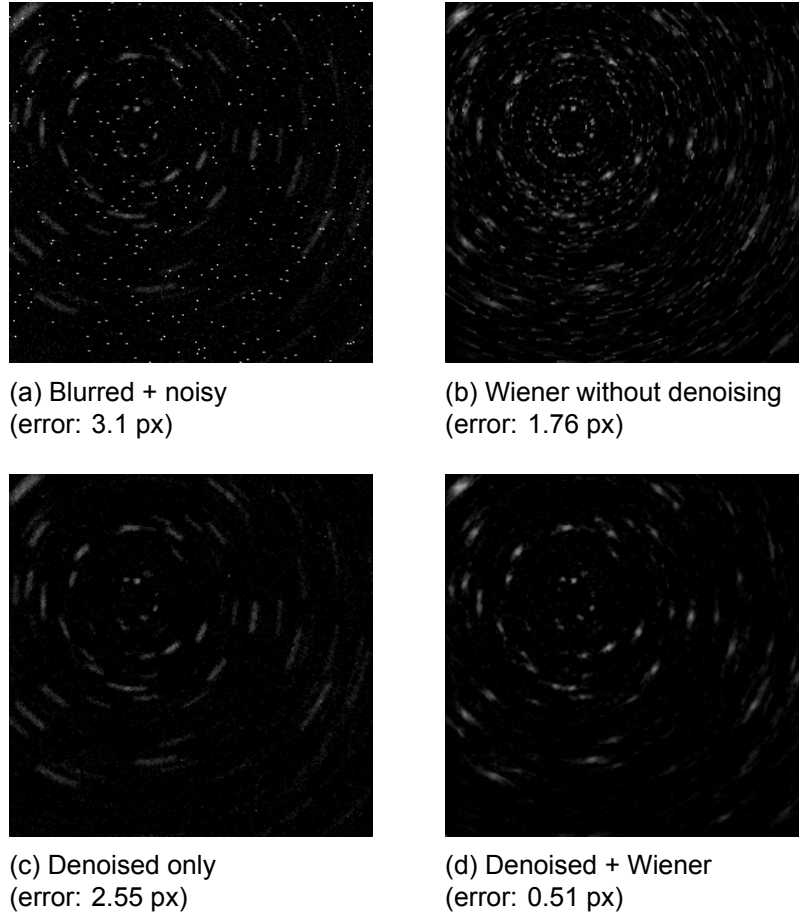


Figure 6.3: Effect of denoising on deblurring performance.

It is clearly visible from Figures 6.3a and 6.3c that the salt and pepper noise is effectively suppressed by the denoising algorithm.

Without denoising, Wiener deconvolution yields a centroiding error of 1.76 pixels. After denoising, this error decreases to 0.51 pixels. Even when applied on its own, denoising reduces the centroiding error from 3.1 pixels to 2.55 pixels. These results show that noise removal plays a key role in improving centroiding accuracy and enhances the effectiveness of the subsequent deblurring step.

6.2.4 Star Detection

Figure 6.4 compares the number of detected stars before and after restoration with the Wiener filter. The images correspond to thresholded versions of the degraded and restored images shown in Figures 6.3a and 6.3d respectively.

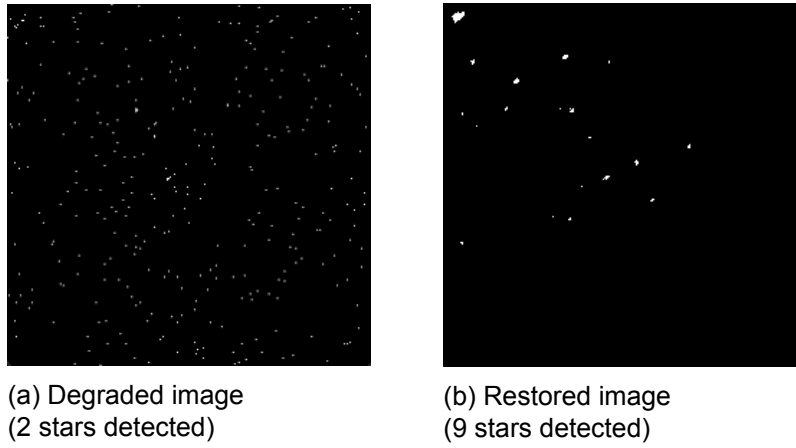


Figure 6.4: Thresholded images before and after restoration.

Only two stars are detected in the degraded image, whereas nine stars are detected after restoration. Additional white pixels were rejected during processing because they were classified as noise or located too close to the image boundary. This comparison illustrates the improvement in effective SNR and the reconstructive capability achieved through denoising and deblurring.

6.2.5 Benchmarking

Algorithm execution times were measured in Python. Figure 6.5 shows the runtime of a full Wiener deconvolution, a single RL iteration and the denoising algorithm.

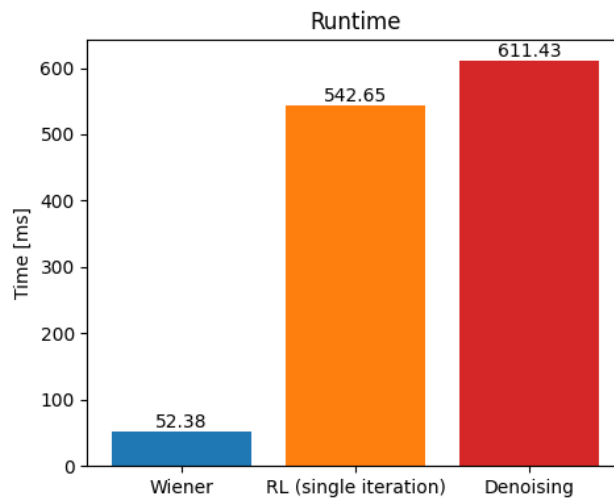


Figure 6.5: Runtime of Wiener, RL and denoising methods

The denoising algorithm required 611 ms. A single RL iteration required 542 ms, while the complete Wiener deconvolution required 52 ms. RL is therefore significantly more computationally demanding, as it frequently requires more than ten iterations.

Additional experimental results are provided in Appendix B.

6.3 Discussion

For the degraded and denoised cases (Figures 6.3a and 6.3c), a lower threshold was used during star detection; otherwise, too few stars were identified, as illustrated in Figure 6.4a. Although centroid computation is still possible with fewer detections, it is not practically useful, since at least four stars are required for attitude determination, as discussed in Section 3.2.1.

The choice of the constant K in the Wiener filter (Equation 4.13) proved challenging. A value that performed well for one image did not necessarily yield good results for another, and in some cases the algorithm became unstable for certain choices of K . In this respect, the RL algorithm demonstrated greater robustness, as it does not rely on a single global regularization parameter in the same way.

The runtime comparison shown in Figure 6.5 was obtained using Python implementations. The Wiener filter was largely implemented using optimized library routines (e.g., FFT operations in NumPy), whereas the RL and denoising algorithms relied on custom pixel-wise iterations. As a result, the observed runtime difference may be somewhat amplified by implementation details. Nevertheless, even with optimized code, RL is expected to remain computationally more demanding, as it is inherently iterative, in contrast to the closed form Wiener filter.

All experiments assumed that the rotation axis passes through the optical center. In practical scenarios, this assumption may not hold, and more general motion models would be required to accurately describe the image formation process.

Overall, the results demonstrate that denoising and deblurring can significantly improve centroiding accuracy and increase the number of detectable stars in degraded images. However, the computational requirements of the investigated methods may exceed the real time constraints and limited processing capabilities of typical star tracker systems. While the potential benefits are clear, future work should therefore focus on more computationally efficient alternatives better suited for embedded implementations.

7 Coning Motion Analysis and Validation

This section evaluates the attitude propagation methods presented in Section 5.2 in terms of accuracy under coning motion. The simple kinematic integration, rotation vector update and direct quaternion update methods are compared using both simulation and experimental data collected with the DTU star tracker.

Simulations are used to explore the influence of different coning parameters and noise levels under controlled conditions, while experimental measurements provide insight into the behavior of the algorithms when applied to real sensor data. The main objective of this chapter is to assess whether the simple kinematic integration method currently implemented on the DTU star tracker can be improved by alternative approaches, while remaining within the operational and computational constraints of the system.

7.1 Methodology

The experimental evaluation was conducted using a star tracker (DTU's Mikro Advanced Stellar Compass, μ ASC) equipped with a built-in inertial measurement unit (IMU), as well as an external IMU mounted alongside the star tracker. Both units were based on the ASM330LHH MEMS inertial sensor [24]. In this work, only the gyroscope measurements from each IMU were used. The star tracker IMU provided angular rate measurements at an output rate of 80 Hz and has a specified angular random walk of $0.14^\circ/\sqrt{h}$. The external IMU, using the same sensor model, was polled at 3333 Hz and has a specified angular random walk of $0.21^\circ/\sqrt{h}$. In addition, the star tracker provided absolute attitude estimates at an output rate of 4 Hz. The external IMU was included to examine whether a higher sampling rate leads to any measurable improvement in the evaluated attitude propagation methods.

The experimental setup was mounted on a three degree of freedom turntable configured to perform yaw–pitch–roll rotations, corresponding to the Z–Y–X Euler angle convention described in Section 2.2.3, and is shown in Figure 7.1a. The turntable motion was controlled using three PI C-863 Mercury Servo Controllers [25]. The star tracker was rigidly attached to the turntable such that its body frame axes were aligned with the turntable axes. The external gyroscope was mounted in close proximity to the star tracker, with its axes aligned, as illustrated in Figure 7.1b.

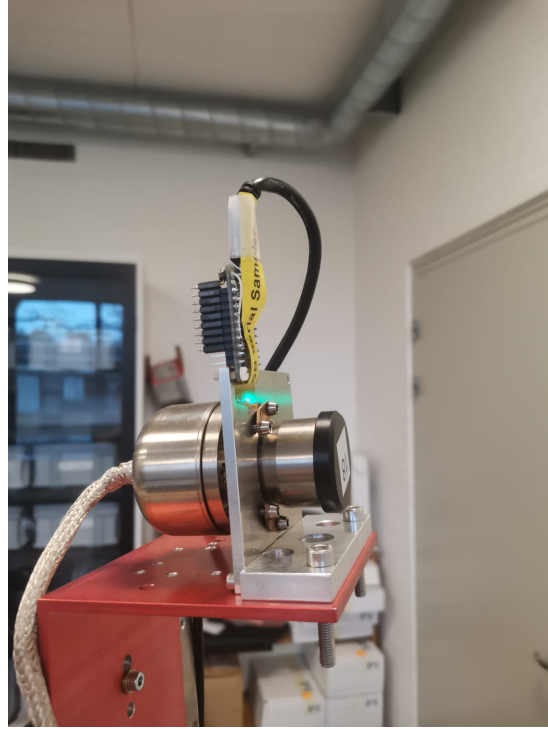
Angular rate measurements from both IMUs were recorded together with the star tracker attitude solution. The star tracker's built-in IMU had been previously calibrated, while the bias of the external gyroscope was estimated and removed during the approximately first ten seconds of data from each experimental run. The operational limitations of the star tracker were taken into account, in particular its maximum supported rotational speed of approximately $6^\circ/s$.

A series of experiments were conducted with varying rotational speeds and coning parameters. The commanded coning motions were generated using analytical coning equations described in Section 5.1.1. The star tracker provides multiple data outputs, of which the following were used in this study:

- an attitude estimate computed from star images and the onboard star catalog,
- a validity flag indicating the availability of a reliable star solution.



(a) Three-axis turntable setup



(b) Star tracker and external gyroscope

Figure 7.1: Experimental setup used for the coning motion experiments.

Due to its high accuracy, the star tracker attitude solution was used as the reference attitude for error evaluation whenever a valid measurement was available. All experimental data were collected on November 6, 2025, on the roof of DTU Space under (mostly) clear night sky conditions.

7.2 Simulation

Simulations were first conducted to evaluate the three attitude propagation methods under controlled conditions. For a given set of coning parameters, defined by the half-angle α and the angular rate ω of the rotation vector, reference solutions were generated using analytical coning formulation of (5.5). These reference solutions were treated as ground truth and compared against the attitudes obtained using the different attitude propagation methods.

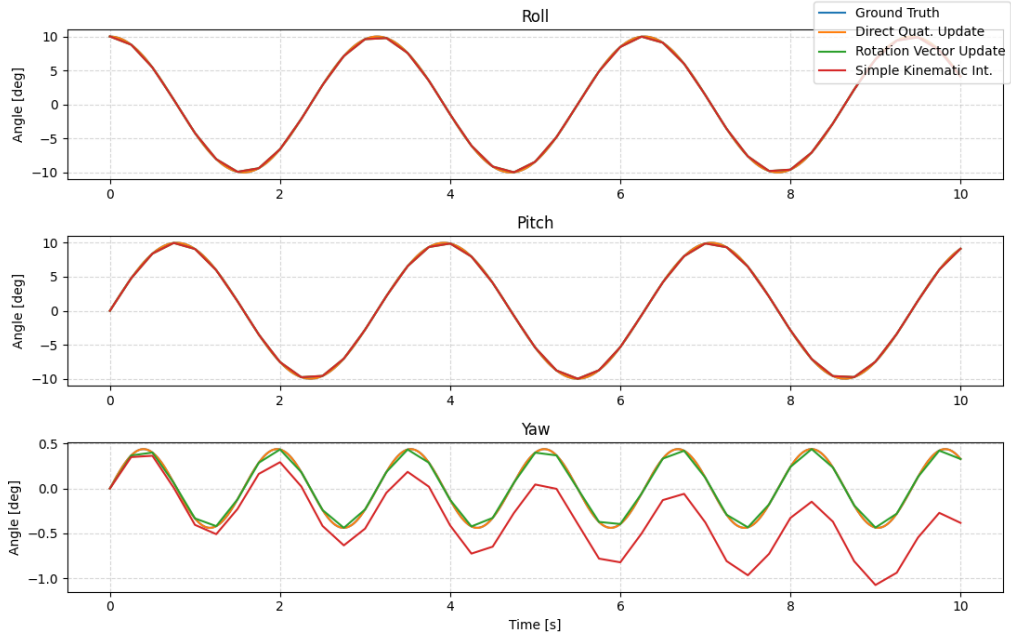
The two-speed methods, namely simple kinematic integration and rotation vector update, were simulated with a sensor sampling rate of 80 Hz, while the attitude update was performed at 4 Hz to match the star tracker attitude output rate and the gyroscope rate used in the experimental setup. This meant that attitude updated was performed after 20 sensor measurements. In contrast, the direct quaternion update was applied at the sensor rate.

Error Growth Metric

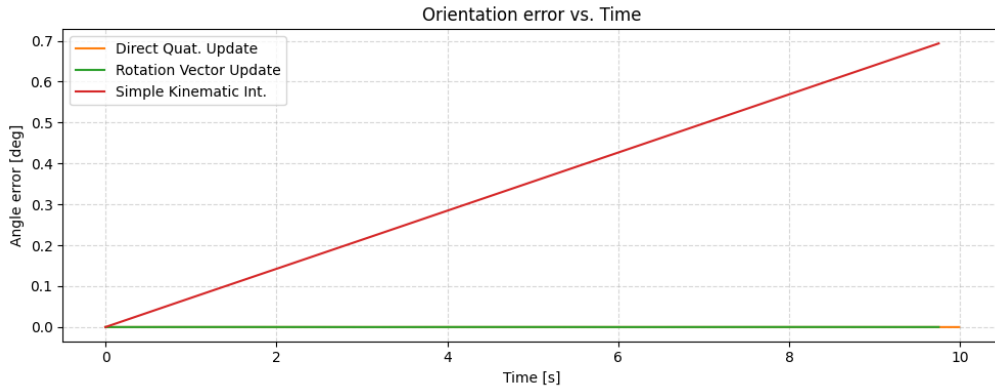
For each propagation method, the attitude error was computed with respect to the ground truth using the error quaternion given in (2.24). The time history of the attitude error was then used to estimate an error growth rate, hereafter referred to as the *error slope*, which serves as a quantitative measure of long term error accumulation under coning motion. The error slope was obtained by approximating the error evolution with a first

order polynomial in time.

An example simulation result for $\alpha = 10^\circ$ and $\omega = 2$ rad/s is shown in Figures 7.2a and 7.2b. The first figure shows the estimated attitude obtained using each propagation method, while the second shows the corresponding attitude error relative to the ground truth. The discrete appearance of the curves for the two-speed methods reflects the lower attitude update rate of 4 Hz.



(a) Estimated attitude.



(b) Attitude error relative to the analytical ground truth.

Figure 7.2: Simulation results for $\alpha = 10^\circ$ and $\omega = 2$ rad/s.

Error Growth Across Coning Parameter Space

While the example above illustrates the behavior of the propagation methods for a single coning case, a broader assessment requires examining how the error evolves across a range of coning motions. The simulation was extended by systematically varying the half-angle α and the angular rate ω of the rotation vector and computing the corresponding error slope for each parameter combination.

With the operational constraint of the star tracker in mind, and using the analytical expres-

sion for the angular rate magnitude given in (5.7), a subset of half-angles $\alpha \in [1^\circ, 20^\circ]$ and angular velocities $\omega \in [0.1, 1]$ rad/s was selected. This parameter range results in maximum angular rates of approximately $20^\circ/\text{s}$. For each (α, ω) pair, the error slope was evaluated using the procedure described above.

Figure 7.3 summarizes the resulting error slope as a function of α and ω for the three propagation methods in the absence of measurement noise. Figures 7.3a, 7.3b, and 7.3c correspond to the direct quaternion update, rotation vector update, and simple kinematic integration, respectively.

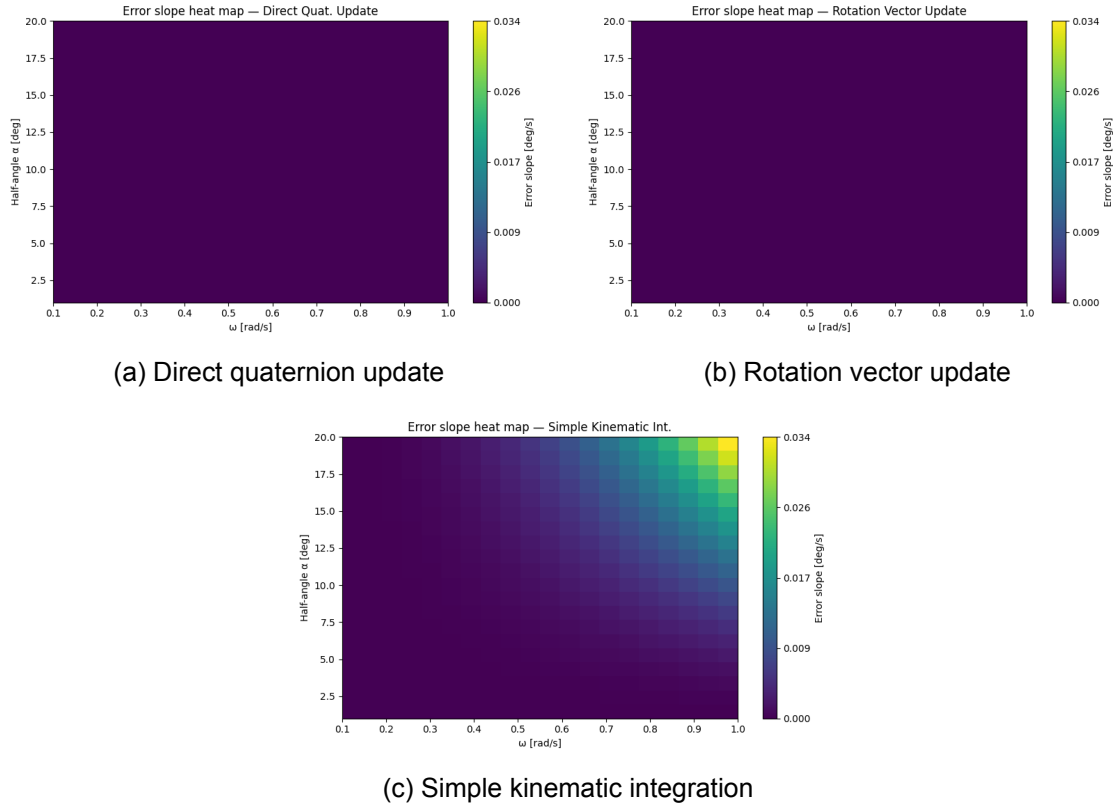


Figure 7.3: Error slope heatmaps without measurement noise as a function of half-angle α and angular velocity ω .

In the case without noise, the differences between the algorithms are clearly visible, especially for larger angular rates. Coning effects lead to significant error growth for simple kinematic integration, while both the rotation vector update and the direct quaternion update substantially reduce this error.

To assess the influence of realistic sensor noise, the same simulations were repeated with additive white Gaussian noise applied to the gyroscope measurements, consistent with the specified angular random walk of the star tracker IMU ($0.14^\circ/\sqrt{\text{h}}$, corresponding to a standard deviation of $0.0209^\circ/\text{s}$).

Furthermore, the average error slope was computed for each method by considering only parameter combinations with angular rates below $6^\circ/\text{s}$. Figure 7.4 shows the resulting comparison for both simulations with and without noise at an attitude update rate of 4 Hz.

When measurement noise is not included, both the direct quaternion update and the rotation vector update exhibit negligible error growth, whereas simple kinematic integration

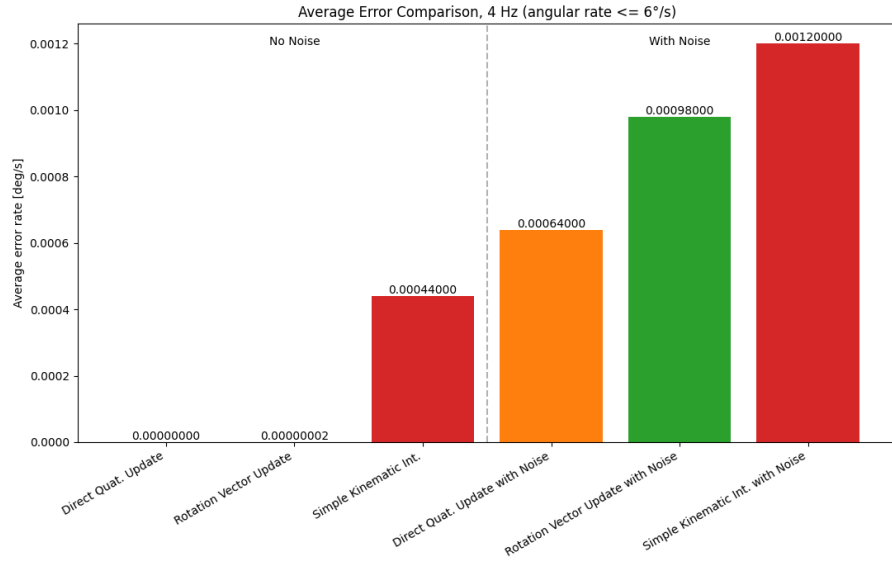


Figure 7.4: Average error slope for angular rates $\leq 6^\circ/\text{s}$ with and without measurement noise at a 4 Hz attitude update rate.

shows a clearly higher error slope. With noise present, all methods experience a comparable increase in error, and the performance gap between them narrows, indicating that sensor noise has a significant influence on the error behavior at low angular rates.

Sensitivity to Attitude Update Frequency

Finally, the sensitivity of the two-speed algorithms to the attitude update frequency was investigated. Figure 7.5 shows the error slope heatmaps for the rotation vector update at update rates of 4 Hz and 0.2 Hz, while Figure 7.6 shows the corresponding results for simple kinematic integration. A logarithmic color scale is used to emphasize relative differences in error growth across the parameter space. The direct quaternion update is not included in this analysis, as it is a single speed algorithm that operates at the sensor sampling rate and does not depend on a separate attitude update frequency.

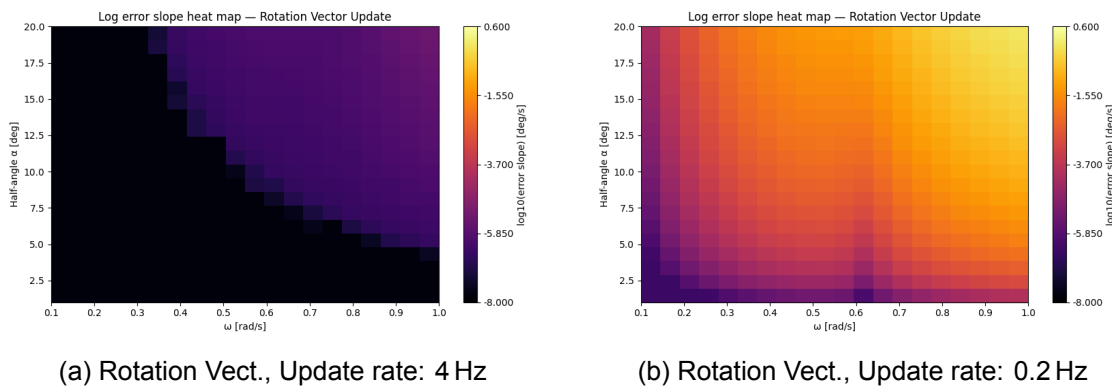


Figure 7.5: Error slope heatmaps for the rotation vector update at different attitude update frequencies.

The corresponding average error slopes without measurement noise are summarized in Figure 7.7. Even for angular rates below $6^\circ/\text{s}$, less frequent attitude updates lead to increased error accumulation. This effect is most pronounced for simple kinematic inte-

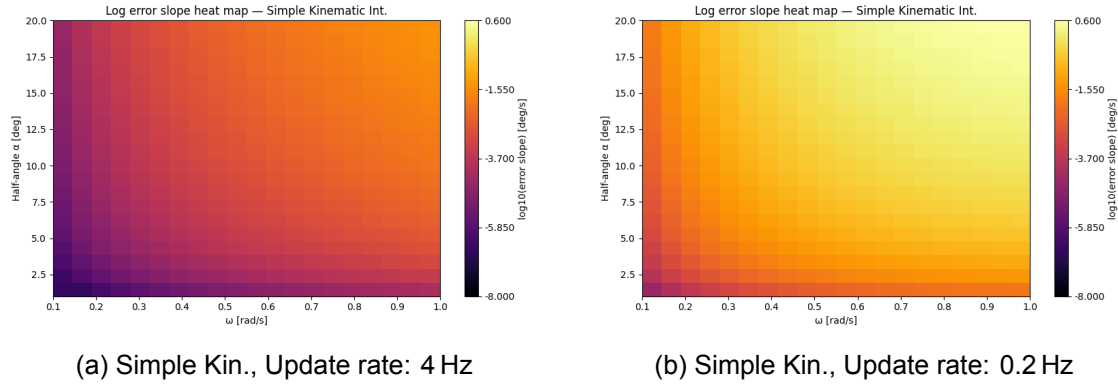


Figure 7.6: Error slope heatmaps for simple kinematic integration at different attitude update frequencies.

gration, for which coning errors accumulate in the rotation vector approximation between successive attitude updates. The rotation vector update exhibits a weaker dependence on update frequency because it explicitly compensates for coning effects, reducing the rate at which errors build up over the update interval.

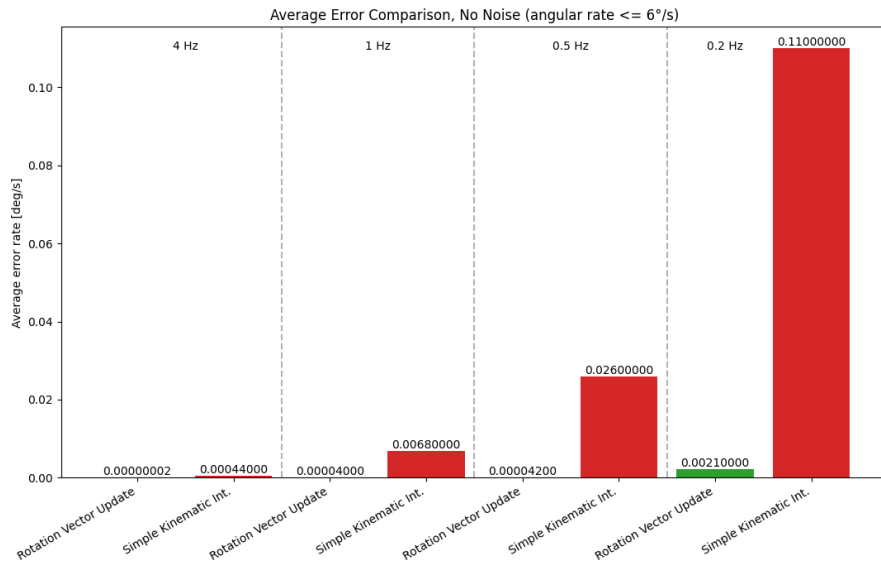


Figure 7.7: Average error slope as a function of attitude update frequency without measurement noise.

7.3 Experimental Results

A total of 17 valid experimental runs were conducted, for which synchronized data from the star tracker and both inertial measurement units were successfully recorded. Of these runs, nine corresponded to pure coning motion, while the remaining eight involved simpler rotational maneuvers, such as rotation about a single axis or a single change in the rotation axis. Data from both the built-in IMU of the star tracker and the external IMU were used to evaluate the three attitude propagation methods.

For each experiment, the gyroscope measurements were converted into attitude updates using the respective propagation method. The star tracker attitude estimates were treated

as reference points, serving as anchors for the propagation process. Specifically, the most recent valid star tracker attitude was used as the initial orientation, onto which the integrated gyroscope measurements were applied. This procedure was repeated independently for each valid star tracker measurement.

Two different propagation strategies were evaluated for the two-speed algorithms. In the first case, attitude updates were performed at regular intervals corresponding to every 20 gyroscope samples, independent of the availability of valid star tracker measurements. In the second case, attitude updates were performed only when a new valid star tracker solution became available. A schematic illustration of these two approaches is shown in Figure 7.8.

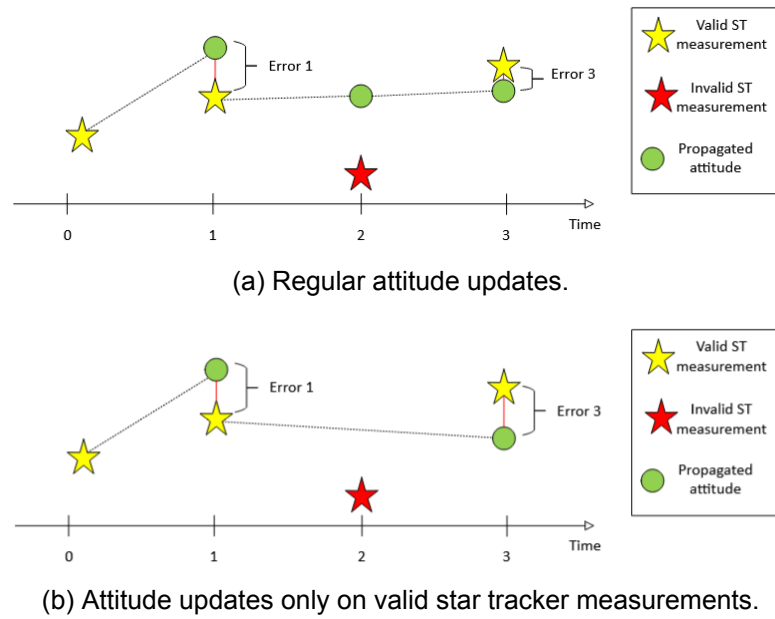


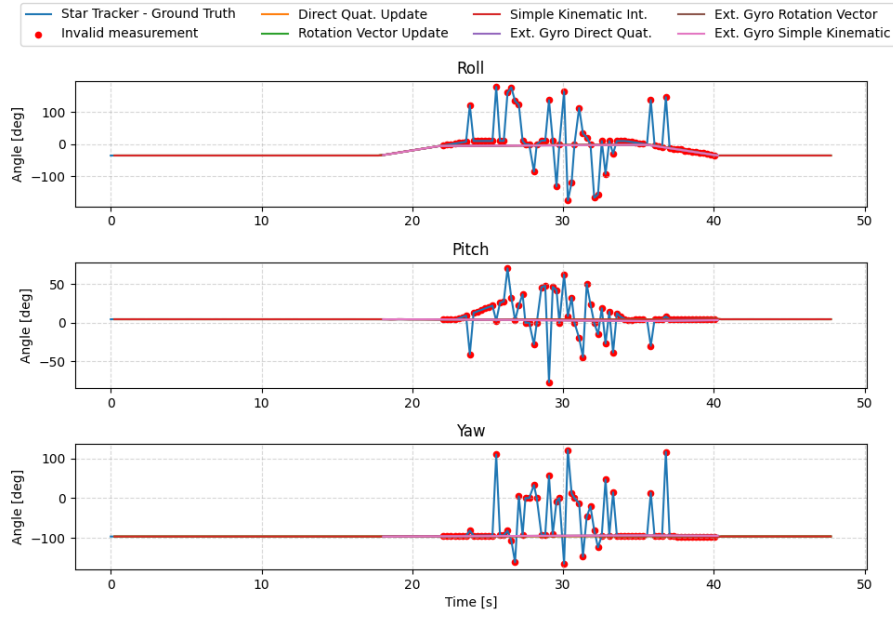
Figure 7.8: Schematic illustration of the two attitude propagation strategies evaluated in the experiments.

The propagated attitude estimates were compared against the corresponding star tracker measurements by computing the error quaternion at each valid star tracker time step. The resulting errors were accumulated over time to form an error history, and an average error rate was computed by dividing the final accumulated error by the elapsed time.

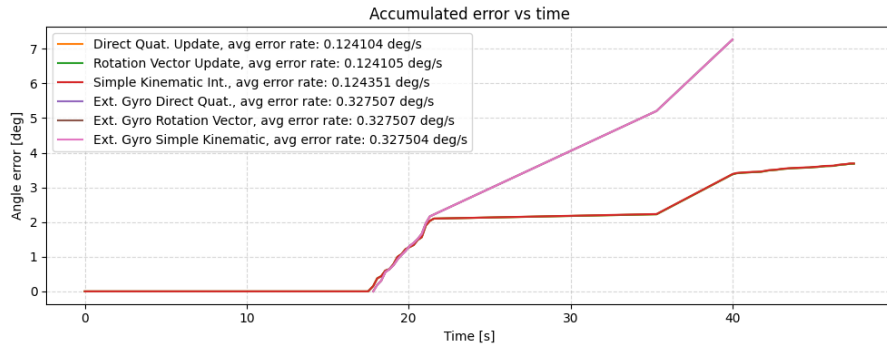
The following results illustrate the difference between the two attitude propagation strategies described above. Two representative scenarios with a high number of invalid star tracker measurements are considered. In the first scenario, the system executes a simple rotational maneuver consisting of a rotation about the x -axis followed by an immediate rotation about the y -axis and back. In the second scenario, a high frequency coning motion is performed.

Figures 7.9 and 7.10 show that when attitude updates are performed at regular intervals, the differences between the propagation methods are negligible. In contrast, when attitude updates are restricted to valid star tracker measurements only, differences become apparent once the rotation axis changes. In this case, propagation methods that account for coning effects accumulate noticeably less error than simple kinematic integration.

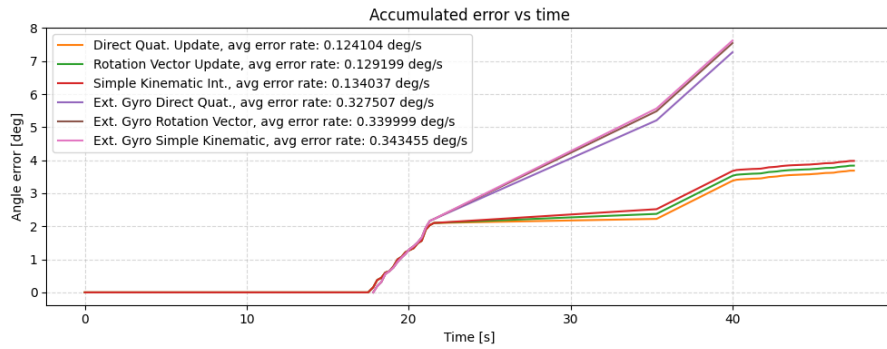
To obtain a more general assessment of the experimental results, the average error rates



(a) Estimated attitude

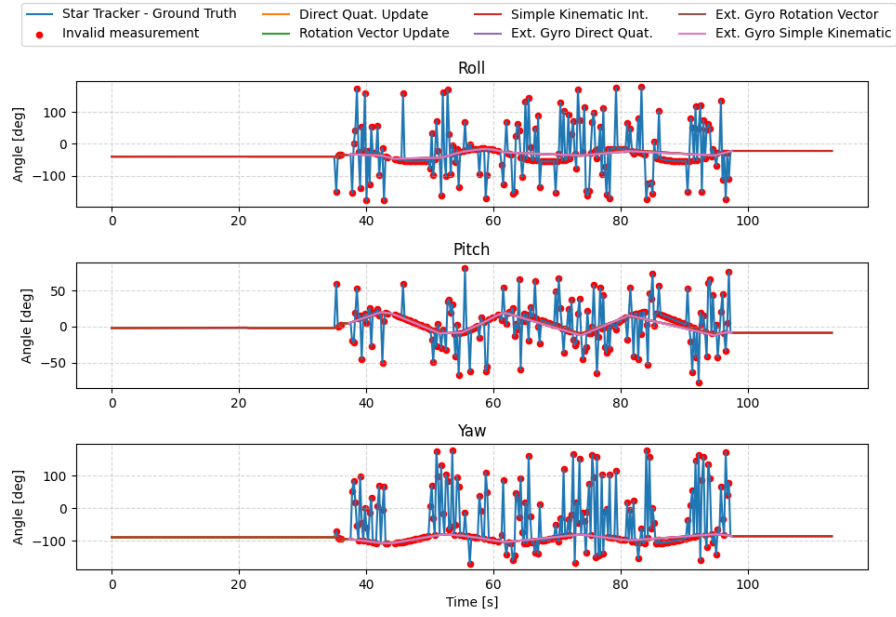


(b) Accumulated error (regular updates)

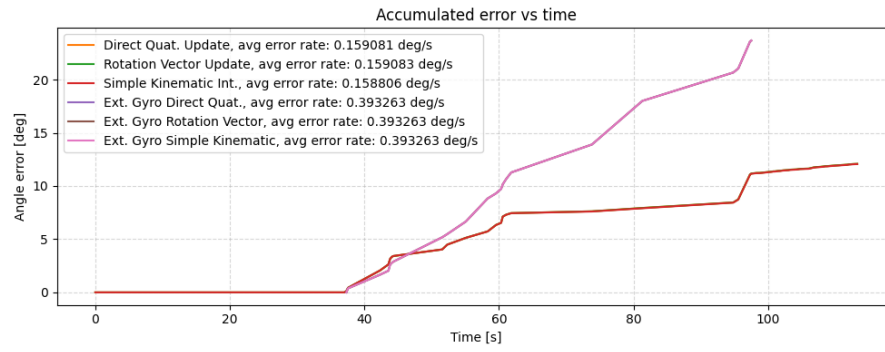


(c) Accumulated error (valid-only updates)

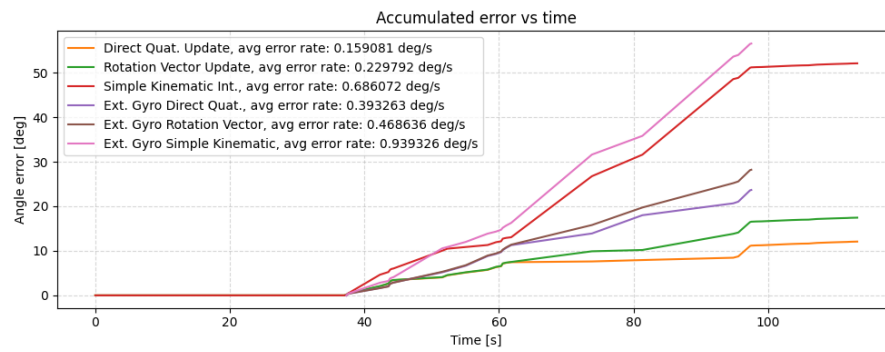
Figure 7.9: Simple rotational maneuver consisting of a rotation about the x -axis followed by a rotation about the y -axis and back ($\omega = 0.16$ rad/s).



(a) Estimated attitude



(b) Accumulated error (regular updates)

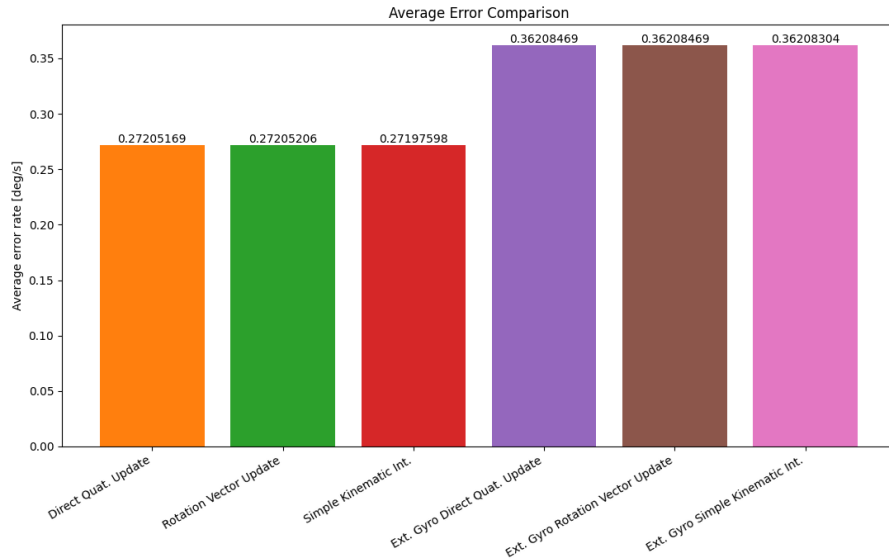


(c) Accumulated error (valid-only updates)

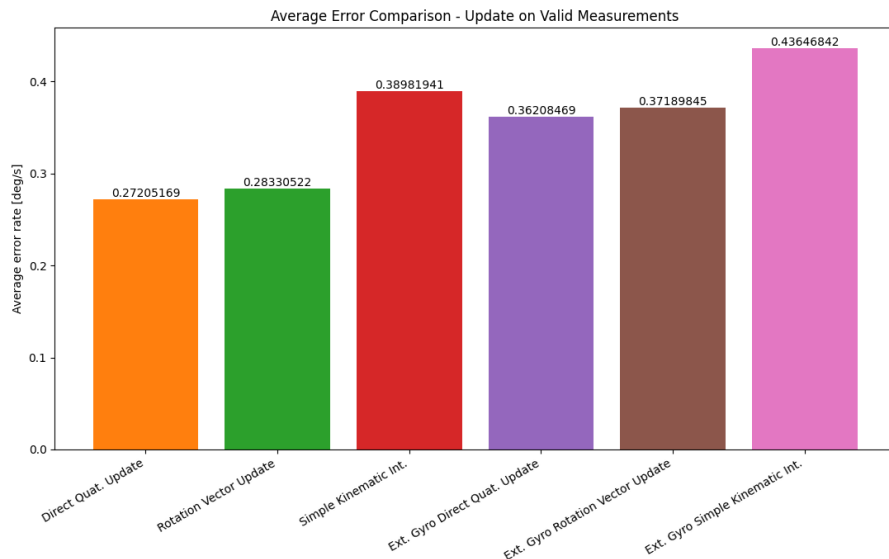
Figure 7.10: High frequency coning motion with $\alpha = 20^\circ$ and $\omega = 0.25$ rad/s.

were computed for each experiment and then combined into a weighted average over all 17 experimental runs. This provides an aggregate measure of performance for each attitude propagation method. Additional individual experiment results are provided in the appendix D.

The resulting average error rates are summarized in the bar charts shown in Figure 7.11. Figure 7.11a corresponds to the case where attitude updates were performed regularly, every 20 sensor measurements, while Figure 7.11b shows the case where attitude updates were performed only when valid star tracker measurements were available.



(a) Regular attitude updates every 20 sensor measurements.



(b) Attitude updates only on valid star tracker measurements.

Figure 7.11: Average experimental error rates for the evaluated attitude propagation methods.

The bar charts in Figure 7.11 show that when attitude updates are performed at regular intervals, there is no significant difference in performance between the evaluated atti-

tude propagation methods. In contrast, when attitude updates are restricted to valid star tracker measurements only, clear differences emerge. Since the direct quaternion update operates as a single-speed algorithm, its average error remains unchanged between the two cases. The rotation vector update exhibits a modest increase in error, while simple kinematic integration shows a pronounced increase in average error rate. This behavior is consistent with the simulation results presented in Figure 7.7, where infrequent attitude updates were shown to lead to increased coning error accumulation.

7.4 Discussion

Figure 7.11 shows that the external IMU consistently exhibited larger attitude errors than the built-in IMU of the star tracker. This difference is primarily attributed to mechanical and calibration factors. The external IMU was hand mounted on a separate PCB and attached to the star tracker, making it more susceptible to axis misalignment. In addition, bias and noise mitigation was limited, as the bias was estimated only from the first ~ 10 s of data prior to each experiment. These factors likely account for the increased error observed in the external IMU results.

Although the external IMU was originally included to investigate the effect of higher sampling rates on coning error compensation, this objective could not be fully achieved due to excessive noise and drift. Nevertheless, the external IMU exhibited the same qualitative behavior as the built-in IMU: negligible differences between propagation methods when attitude updates were performed regularly, and increased sensitivity to coning effects when updates were restricted to valid star tracker measurements only. This consistency supports the general conclusions drawn from the experimental results.

A mismatch is observed between the absolute error slopes obtained in simulation (Figure 7.4) and those measured experimentally (Figure 7.11), even when measurement noise is included in the simulations. However, the relative trends are consistent. In particular, the order of magnitude increase in error between simple kinematic integration and rotation vector update for decreasing attitude update frequencies in simulation (Figure 7.7) closely matches the increase observed experimentally when transitioning from regular updates to updates based only on valid star tracker measurements.

One likely contributor to the higher experimental error levels are implementation differences between the processing performed in this work and the internal star tracker processing. Initial difficulties encountered when aligning IMU and star tracker measurements based on timestamps suggest that some residual mismatches may remain. Since access to the star tracker source code was not available, full consistency between the two processing chains could not be verified, and minor implementation level discrepancies may therefore contribute to the observed errors.

The high number of invalid star tracker measurements can be explained by the experimental conditions. Many tests were conducted near the operational rotational speed limit of the star tracker, and environmental conditions were suboptimal due to light pollution from near full moon conditions and partial cloud coverage.

Overall, both simulation and experimental results indicate that coning errors are negligible within the operational angular rate range of the star tracker, provided that attitude updates are performed at a sufficiently high frequency (4 Hz in this work). When updates are delayed, coning errors accumulate rapidly in simple kinematic integration, while the rotation vector update remains more robust due to its explicit compensation of non-commutativity effects.

Considering both estimation accuracy and computational complexity (Table 5.1), the rotation vector update represents a practical alternative to simple kinematic integration when frequent attitude updates cannot be guaranteed. When updates are frequent, simple kinematic integration performs comparably at lower computational cost. The direct quaternion update, although more accurate in principle, is not well suited for computationally constrained systems due to its substantially higher cost. Hybrid strategies that dynamically switch between methods were considered, but deemed impractical, as the required decision logic is more computationally expensive than the rotation vector update itself.

8 Conclusion and Future Work

This section summarizes the main findings of this work. Detailed discussions of the image restoration and coning compensation results are provided in Sections 6.3 and 7.4, respectively.

Main Conclusions

The main conclusions, based on both simulation studies and experimental validation, can be summarized as follows:

- The investigated denoising and deblurring methods significantly improve centroiding accuracy in degraded star field images and enable the detection of additional stars. However, their computational complexity is too high for real-time implementation on the current star tracker hardware.
- Coning errors arising from simple kinematic integration are largely negligible at low angular rates within the operational range of the star tracker. This observation is supported by the experimental results, where no significant difference in attitude accuracy was observed between the evaluated propagation methods.
- Coning errors are strongly dependent on the attitude update frequency. When attitude updates are performed at a low rate, simple kinematic integration accumulates significantly larger errors compared to the rotation vector update, which explicitly compensates for coning effects.
- Although the direct quaternion update provides the highest theoretical accuracy, it is impractical for the target system due to its higher computational cost. Furthermore, the experimental results do not demonstrate a meaningful improvement over the two-speed methods at the rotational rates considered.

Proposed Improvements

Based on these findings, the following improvements are proposed for the DTU star tracker, listed in order of importance:

- Perform attitude updates at a consistently high frequency, regardless of the availability of valid star tracker measurements. In periods where star solutions are invalid, the attitude should be propagated using gyroscope measurements without delaying the update until the next valid star observation.
- Replace the current simple kinematic integration with the rotation vector update method. This approach offers comparable computational cost while explicitly compensating for coning effects.

Future Work

Several directions for future work are identified:

- Conduct coning experiments using a system capable of operating at moderate to high angular rates, in order to evaluate the performance of the proposed methods outside the operational range of the current star tracker.
- Investigate more computationally efficient image denoising and deblurring techniques suitable for real-time implementation on embedded systems with limited processing resources.

- Extend the image deblurring approach to scenarios where the center of rotation does not coincide with the optical center, potentially by estimating the rotation center using IMU measurements.

Bibliography

- [1] Carl Christian Liebe. “Star Trackers for Attitude Determination”. In: *IEEE Aerospace and Electronic Systems Magazine* 10.6 (1995), pp. 10–16.
- [2] Zhiya Mu et al. “Restoration Method of a Blurred Star Image for a Star Sensor Under Dynamic Conditions”. In: *Sensors* 19 (Sept. 2019), p. 4127. DOI: 10.3390/s19194127.
- [3] Paul G. Savage. “Strapdown Inertial Navigation Integration Algorithm Design Part 1: Attitude Algorithms”. In: *Journal of Guidance, Control, and Dynamics* 21.1 (1998), pp. 19–28.
- [4] F. Landis Markley and John L. Crassidis. *Fundamentals of Spacecraft Attitude Determination and Control*. Space Technology Library. Springer, 2014.
- [5] David H. Titterton and John L. Weston. *Strapdown Inertial Navigation Technology*. 2nd. IEE Radar, Sonar, Navigation and Avionics Series. Institution of Electrical Engineers (IEE), 2004.
- [6] Jack B. Kuipers. “Quaternions and Rotation Sequences”. In: *Geometry, Integrability and Quantization*. Coral Press, 1999, pp. 127–143.
- [7] Evandro Bernardes and Stéphane Viollet. “Quaternion to Euler angles conversion: A direct, general and computationally efficient method”. In: *PLOS ONE* 17.11 (2022).
- [8] Malcolm D. Shuster. “A Survey of Attitude Representations”. In: *The Journal of the Astronautical Sciences* 41.4 (Oct. 1993), pp. 439–517.
- [9] D. Mortari, John Junkins, and Malak Samaan. “Lost-in-Space Pyramid Algorithm for Robust Star Pattern Recognition”. In: (Jan. 2001).
- [10] Carl Leake, David Arnas, and Daniele Mortari. “Non-Dimensional Star Identification”. In: *AIAA/AAS Astrodynamics Specialist Conference* (2020).
- [11] Samuel Pedrotty et al. “Design and Performance of an Open-source Star Tracker Algorithm on Commercial Off-the-shelf Cameras and Computers”. In: Feb. 2020.
- [12] Ting Sun et al. “Motion-blurred star acquisition method of the star tracker under high dynamic conditions”. In: *Opt. Express* 21.17 (Aug. 2013), pp. 20096–20110. DOI: 10.1364/OE.21.020096.
- [13] Weina Zhang, Wei Quan, and Lei Guo. “Blurred Star Image Processing for Star Sensors under Dynamic Conditions”. In: *Sensors* 12 (May 2012), pp. 6712–6726. DOI: 10.3390/s120506712.
- [14] Shiqiang Wang et al. “Motion Blurred Star Image Restoration Based on MEMS Gyroscope Aid and Blur Kernel Correction”. In: *Sensors* 18 (Aug. 2018), p. 2662. DOI: 10.3390/s18082662.
- [15] Wu Xiaojuan and Wang Xinlong. “Multiple blur of star image and the restoration under dynamic conditions”. In: *Acta Astronautica - ACTA ASTRONAUT* 68 (June 2011), pp. 1903–1913. DOI: 10.1016/j.actaastro.2010.12.007.
- [16] NASA. *PIA18376*. <https://images.nasa.gov/details/PIA18376>. Accessed: 2026-02-11. 2014.
- [17] Yu-Wing Tai, Ping Tan, and Michael S. Brown. “Richardson-Lucy Deblurring for Scenes under a Projective Motion Path”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.8 (2011), pp. 1603–1618. DOI: 10.1109/TPAMI.2010.222.
- [18] John E. Bortz. “A New Mathematical Formulation for Strapdown Inertial Navigation”. In: *IEEE Transactions on Aerospace and Electronic Systems* (1971).

- [19] John A. Christian et al. *Runge-Kutta Approximations for Direct Coning Compensation Applying Lie Theory*. 2025.
- [20] L. Huang et al. "A new second-order strapdown attitude algorithm". In: *International Journal of Innovative Computing, Information and Control* 9 (Jan. 2013), pp. 3449–3462.
- [21] Robin B. Miller. "A New Strapdown Attitude Algorithm". In: *Journal of Guidance, Control, and Dynamics* 6.4 (1983), pp. 287–291.
- [22] Itzhack Y. Bar-Itzhack. "Navigation Computation in Terrestrial Strapdown Inertial Navigation Systems". In: *IEEE Transactions on Aerospace and Electronic Systems* (1977).
- [23] Paul G. Savage. "Computational Elements for Strapdown Systems". In: *NATO Research and Technology Organization* (2009).
- [24] STMicroelectronics. *ASM330LHH: Automotive 6-axis IMU: 3-axis accelerometer and 3-axis gyroscope*. <https://www.st.com/en/mems-and-sensors/asm330lhh.html>. Accessed: 2025-01-11. 2023.
- [25] Physik Instrumente (PI). *C-863 Mercury Servo Controller*. <https://www.physikinstrumente.com/en/products/controllers-and-drivers/motion-controllers-drivers-for-linear-torque-stepper-dc-servo-motors/c-863-mercury-servo-controller-900606>. Accessed: 2025-01-11. 2023.
- [26] Kurt A. Motelew. *Quaternion to DCM and Back Again*. Tech. rep. Independent Technical Report, 2014.
- [27] José Luis Blanco Claraco. *A Tutorial on SE(3) Transformation Parameterizations and On-Manifold Optimization*. Tech. rep. Technical Report #012010. Universidad de Málaga, 2022.
- [28] D. M. Henderson. *Euler Angles, Quaternions, and Transformation Matrices: Working Relationships*. Tech. rep. Mission Planning and Analysis Division. NASA Johnson Space Center, 1977.

A Conversions Between Attitude Representations

This appendix outlines the conversions between a unit quaternion ($\mathbf{q} = [q_x \ q_y \ q_z \ q_w]^T = [\mathbf{q}_v \ q_w]^T$) and the direction cosine matrix (DCM), rotation vectors, and Euler angles. Conversions among these latter representations are not detailed, since they are used only as intermediate tools in this work.

Quaternion to DCM

For a unit quaternion, the corresponding *direction cosine matrix* can be obtained by [7]

$$\mathbf{C} = \begin{bmatrix} q_w^2 + q_x^2 - q_y^2 - q_z^2 & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & q_w^2 - q_x^2 + q_y^2 - q_z^2 & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & q_w^2 - q_x^2 - q_y^2 + q_z^2 \end{bmatrix}. \quad (\text{A.1})$$

DCM to quaternion

The conversion from a *direction cosine matrix* to a quaternion can be found in [26] and is omitted from this work for brevity. Note that [26] uses scalar first notation of a quaternion, and defines rotations by $\mathbf{q}'_v = \mathbf{q}^* \odot \mathbf{q}_v \odot \mathbf{q}$, which is why the obtained quaternion by this algorithm needs to be conjugated to stay consistent with (2.21).

Quaternion to rotation vector

Given a unit quaternion, the corresponding rotation vector $\boldsymbol{\vartheta}$ is obtained from the quaternion's rotation angle and axis.

The rotation angle is

$$\phi = 2 \arccos(q_w) \quad (\text{A.2})$$

and the rotation axis is

$$\mathbf{e} = \begin{cases} \frac{\mathbf{q}_v}{\|\mathbf{q}_v\|}, & \|\mathbf{q}_v\| > 0, \\ \mathbf{0}, & \text{otherwise.} \end{cases} \quad (\text{A.3})$$

The rotation vector is then

$$\boldsymbol{\phi} = \phi \mathbf{e} = 2 \arccos(q_w) \frac{\mathbf{q}_v}{\|\mathbf{q}_v\|}. \quad (\text{A.4})$$

Rotation vector to quaternion

To convert a rotation vector $\boldsymbol{\phi}$ into a quaternion $\mathbf{q}$, (2.16) is used.

Quaternion to Euler angles

Given a unit quaternion the corresponding Z-Y-X (yaw–pitch–roll) Euler angles are obtained from the relations below [27].

Roll (rotation about x -axis)

$$\phi = \arctan 2(2(q_w q_x + q_y q_z), 1 - 2(q_x^2 + q_y^2)). \quad (\text{A.5})$$

Pitch (rotation about y -axis)

$$\theta = \arcsin(2(q_w q_y - q_z q_x)), \quad (\text{A.6})$$

Yaw (rotation about z -axis)

$$\psi = \arctan 2(2(q_w q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2)). \quad (\text{A.7})$$

Euler angles to quaternion

Given Z-Y-X (yaw–pitch–roll) Euler angles

$$(\phi, \theta, \psi) = (\text{roll}, \text{pitch}, \text{yaw}),$$

with the half-angle terms defined as

$$c_\phi = \cos\left(\frac{\phi}{2}\right), \quad s_\phi = \sin\left(\frac{\phi}{2}\right), \quad c_\theta = \cos\left(\frac{\theta}{2}\right),$$

$$s_\theta = \sin\left(\frac{\theta}{2}\right), \quad c_\psi = \cos\left(\frac{\psi}{2}\right), \quad s_\psi = \sin\left(\frac{\psi}{2}\right).$$

The corresponding quaternion $\mathbf{q} = [q_x \ q_y \ q_z \ q_w]^T$ is then [28]

$$q_x = s_\phi c_\theta c_\psi - c_\phi s_\theta s_\psi, \quad (\text{A.8})$$

$$q_y = c_\phi s_\theta c_\psi + s_\phi c_\theta s_\psi, \quad (\text{A.9})$$

$$q_z = c_\phi c_\theta s_\psi - s_\phi s_\theta c_\psi, \quad (\text{A.10})$$

$$q_w = c_\phi c_\theta c_\psi + s_\phi s_\theta s_\psi. \quad (\text{A.11})$$

B Image Restoration Additional Results

This appendix presents additional experimental results complementing the analysis in Section 6.

Decoupled Motion Model Without Noise

Figure B.1 shows the performance of the deblurring algorithms for an image degraded using the decoupled motion model with $\omega_x = \omega_y = \omega_z = 1$ rad/s and without added noise.

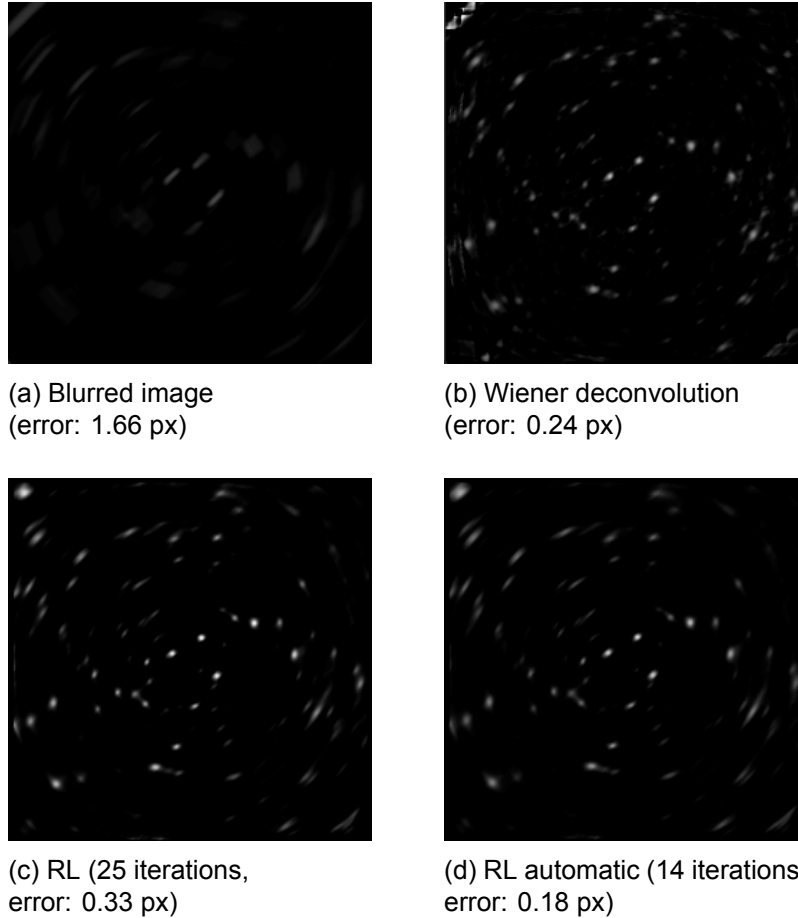
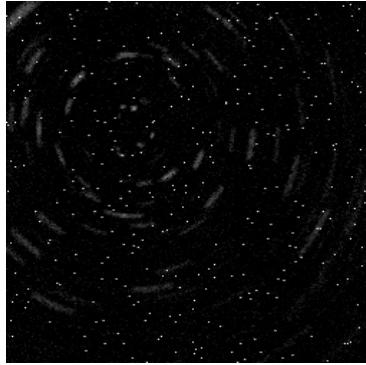


Figure B.1: Deblurring results for the decoupled motion model without added noise.

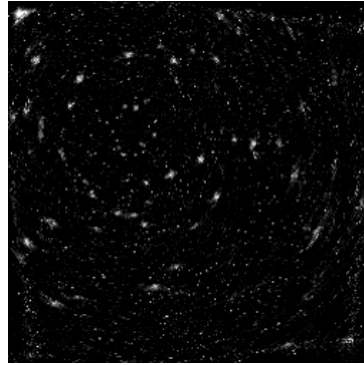
The centroiding error of the blurred image is 1.66 pixels. After restoration, the error decreases to 0.24 pixels using the Wiener filter, 0.33 pixels using RL with 25 iterations, and 0.18 pixels using RL with the automatic termination condition. In all cases, image restoration significantly reduces the centroiding error, demonstrating improved star localization accuracy.

Effect of Denoising on RL Performance

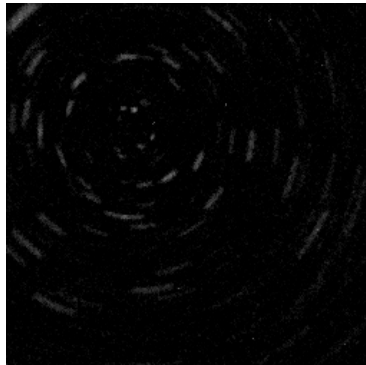
Figure B.2 illustrates the effect of denoising on Richardson–Lucy deconvolution for an image degraded using the coupled motion model with $\omega_x = \omega_y = \omega_z = 1$ rad/s and added noise.



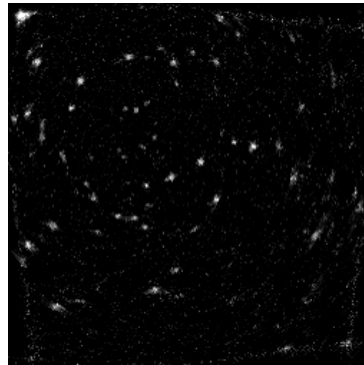
(a) Blurred + noisy
(error: 3.1 px)



(b) RL automatic (15 iterations,
error: 0.71 px)



(c) Denoised only
(error: 2.55 px)



(d) Denoised + RL automatic
(17 iterations, error: 0.61 px)

Figure B.2: Effect of denoising on RL deblurring performance.

For the blurred and noisy image, the centroiding error is 3.1 pixels. Applying RL directly reduces the error to 0.71 pixels. When the image is first denoised, the error decreases from 3.1 to 2.55 pixels, and subsequent RL deblurring further reduces it to 0.61 pixels. These results again demonstrate that denoising improves the effectiveness of the deblurring stage and enhances centroiding accuracy.

C Coning Simulation Results

This appendix presents additional simulation results related to the coning motion analysis discussed in Section 7. The figures included here provide extended visualizations of the error slope behavior across a broader range of coning parameters and attitude update frequencies.

Figure C.1 shows error slope heatmaps for the three attitude propagation methods in the absence of measurement noise, using a logarithmic color scale and an expanded range of angular velocities. The three subfigures correspond to the direct quaternion update (Fig. C.1a), rotation vector update (Fig. C.1b), and simple kinematic integration (Fig. C.1c), respectively.

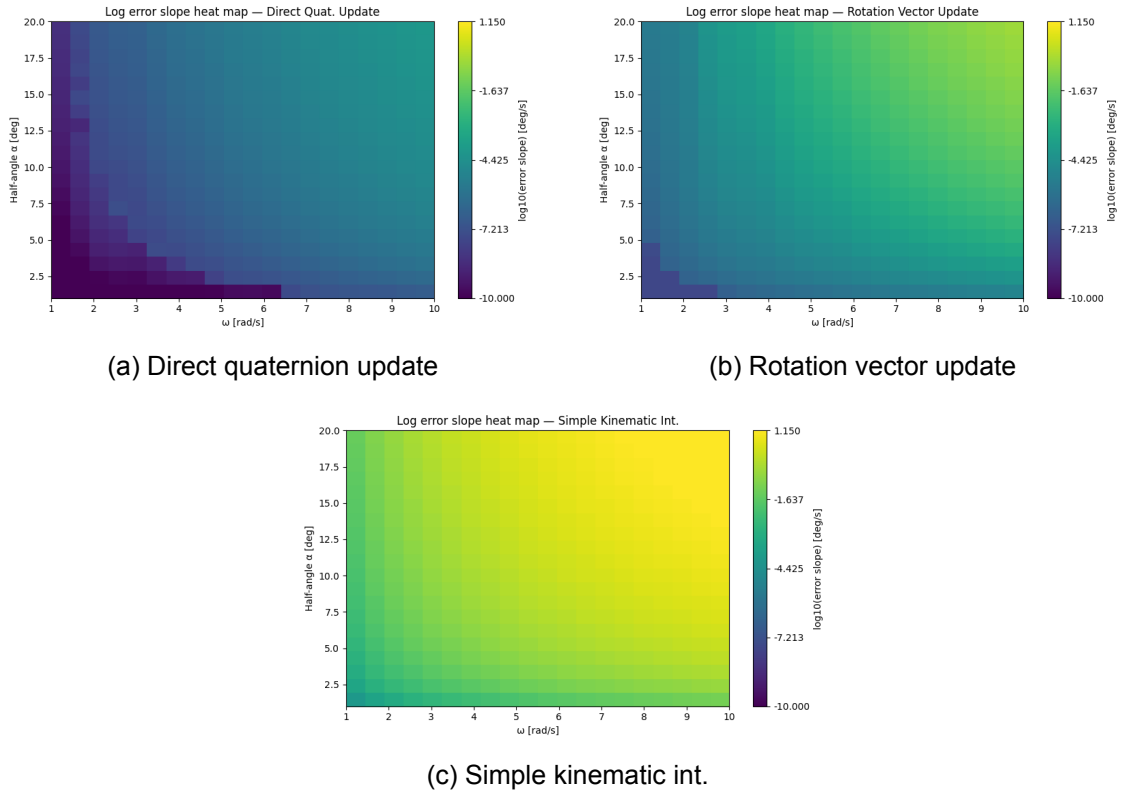


Figure C.1: Error slope heatmaps without measurement noise using a logarithmic color scale and an expanded angular velocity range.

Figure C.2 shows the corresponding error slope heatmaps when additive white Gaussian noise is included in the gyroscope measurements. The subfigures correspond to the direct quaternion update (Fig. C.2a), rotation vector update (Fig. C.2b), and simple kinematic integration (Fig. C.2c).

To further illustrate the sensitivity of the two-speed algorithms to the attitude update frequency, Figures C.3 and C.4 present error slope heatmaps for the rotation vector update and simple kinematic integration, respectively, at update frequencies of 4, 1, 0.5, and 0.2 Hz. A logarithmic color scale is used in all subfigures to highlight relative differences in error growth.

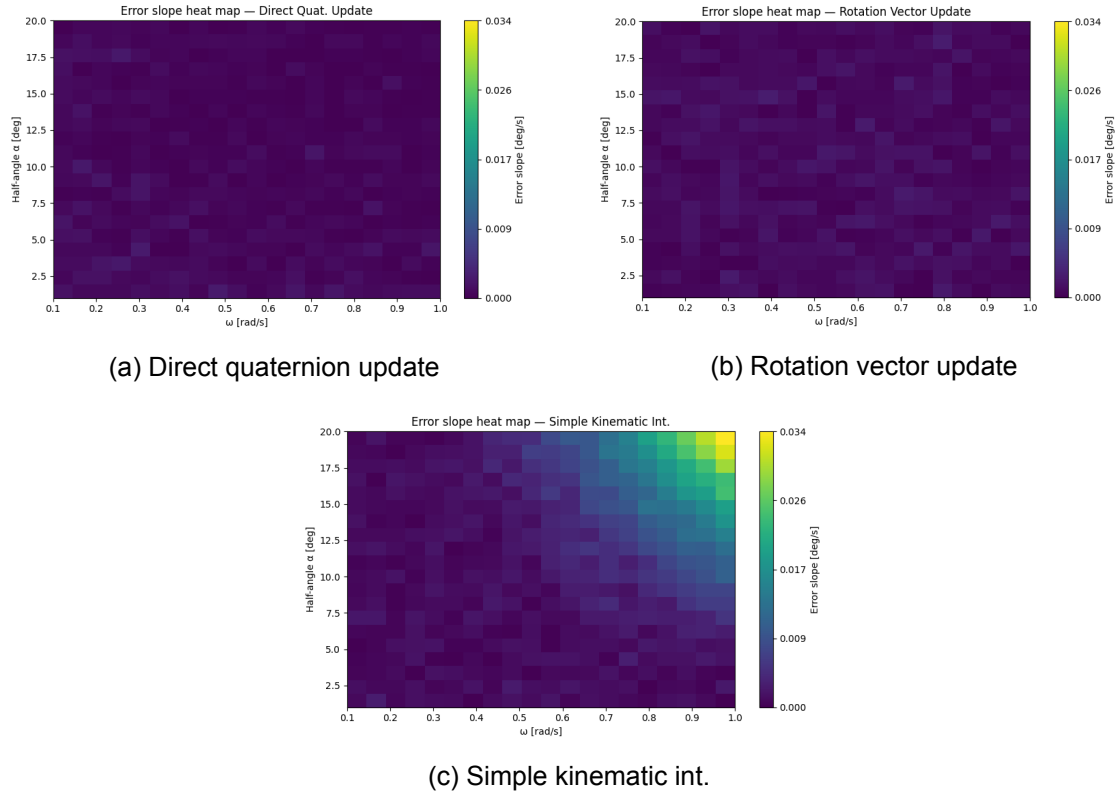


Figure C.2: Error slope heatmaps with measurement noise included.

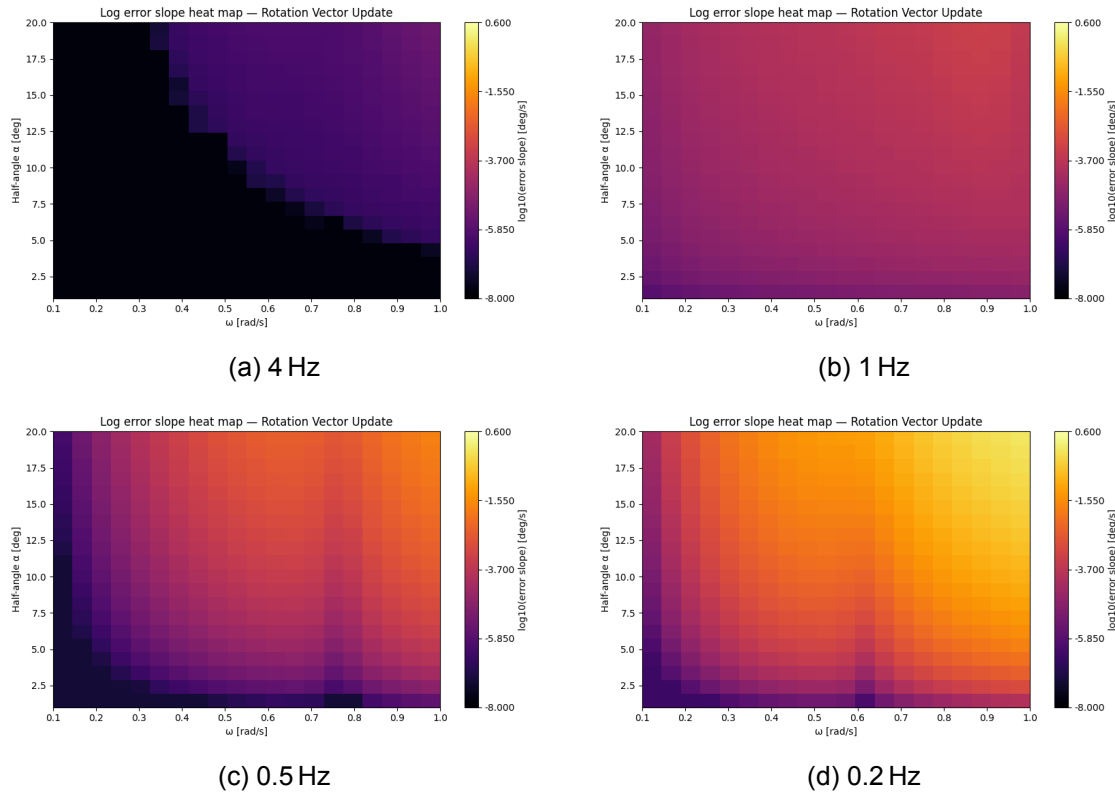
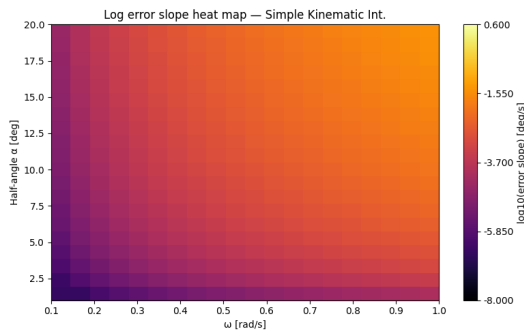
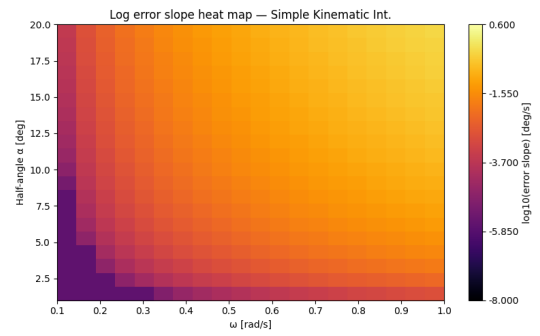


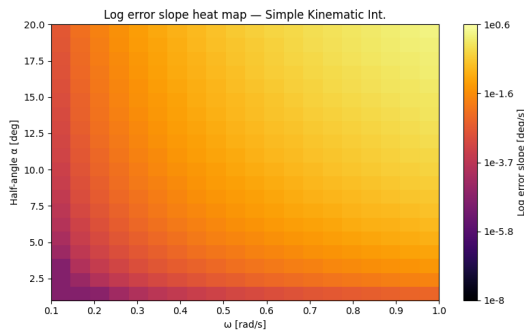
Figure C.3: Error slope heatmaps for the rotation vector update at different attitude update frequencies.



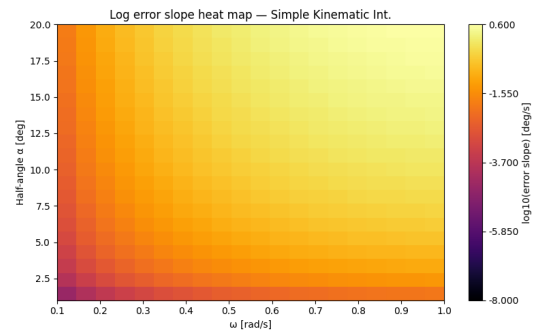
(a) 4 Hz



(b) 1 Hz



(c) 0.5 Hz



(d) 0.2 Hz

Figure C.4: Error slope heatmaps for simple kinematic integration at different attitude update frequencies.

Finally, Figure C.5 summarizes the average error slope for angular rates below $6^\circ/\text{s}$ as a function of attitude update frequency when measurement noise is included.

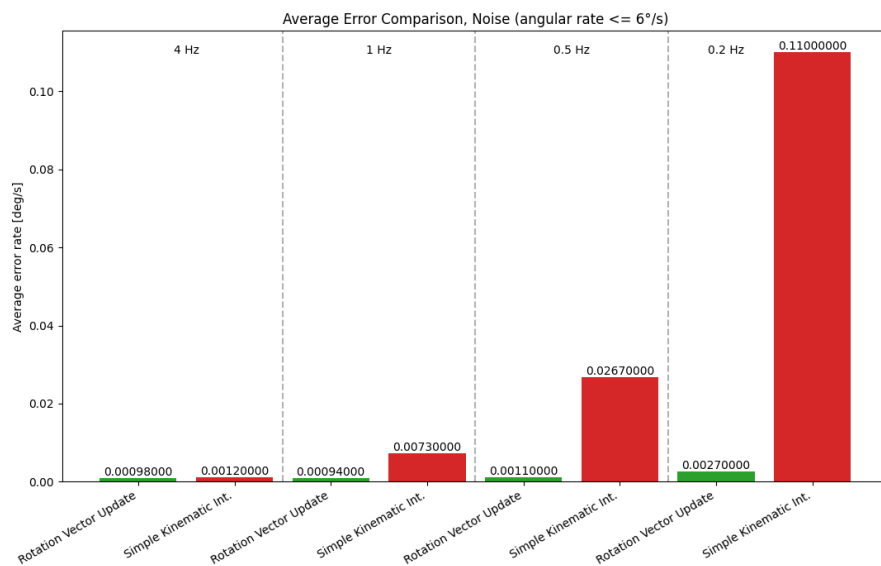
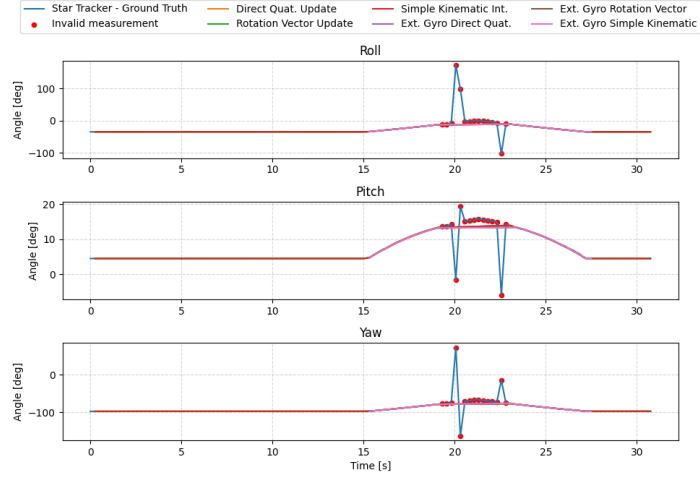


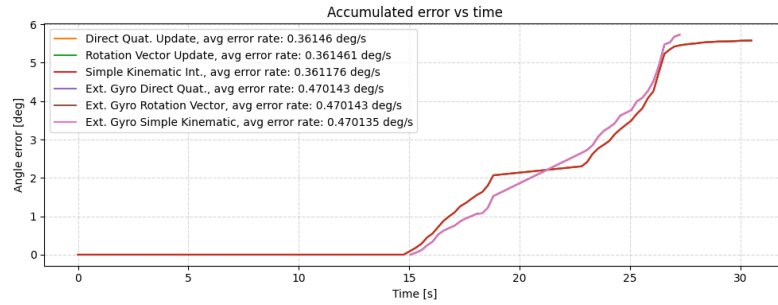
Figure C.5: Average error slope as a function of attitude update frequency for angular rates $\leq 6^\circ/\text{s}$ with measurement noise included.

D Coning Experiment Results

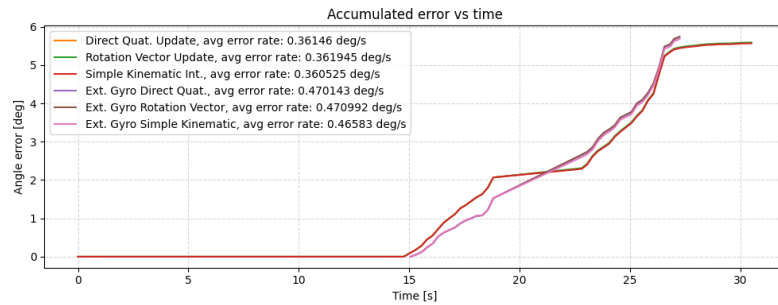
This appendix presents selected experimental results from the coning experiments conducted in this work. The experiments shown here complement the aggregate results discussed in Section 7.3.



(a) Estimated attitude

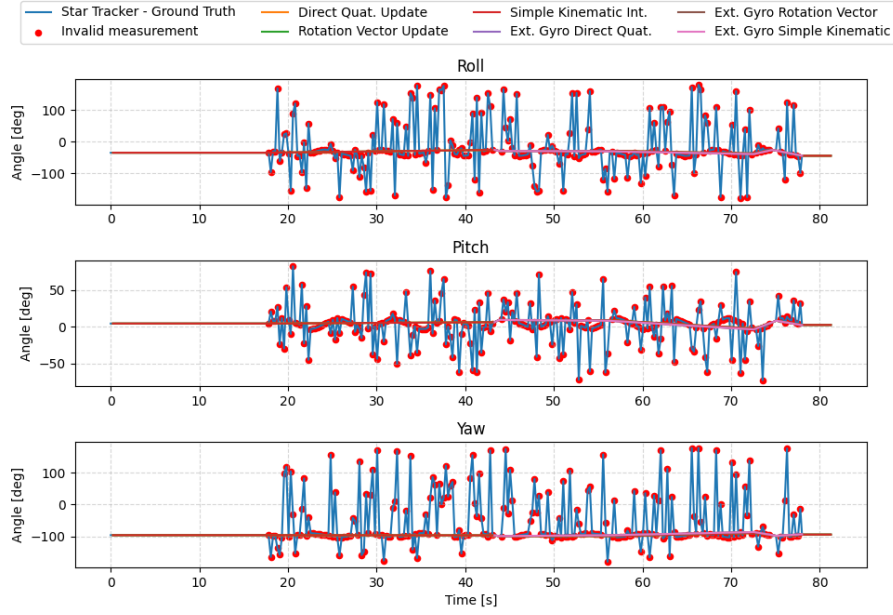


(b) Accumulated error (regular updates)

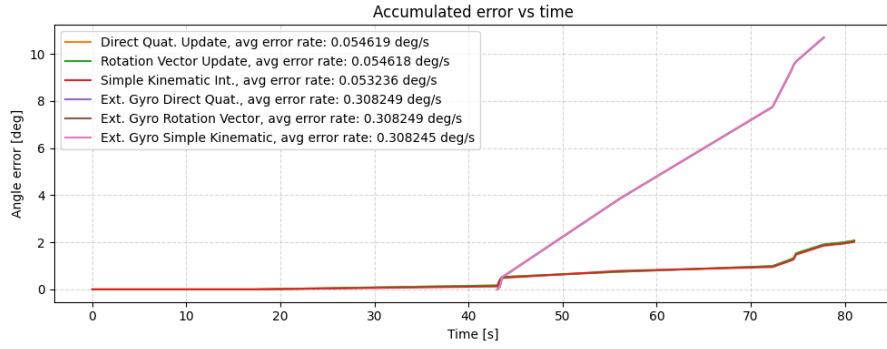


(c) Accumulated error (valid-only updates)

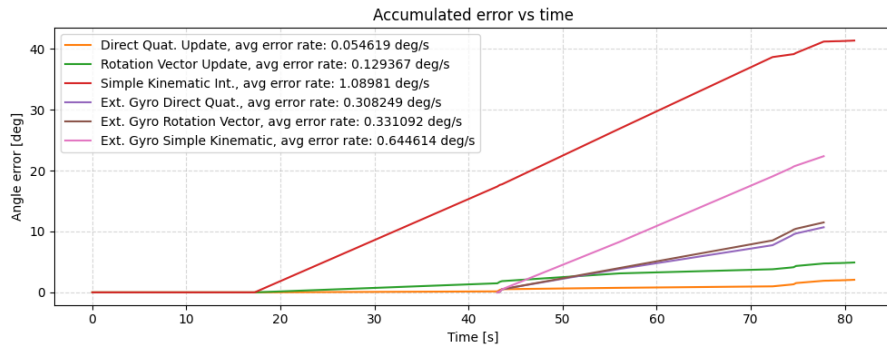
Figure D.1: Rotation about a fixed axis $\mathbf{u} = [1, 1, 1]^T$ with $\omega = 0.16$ rad/s.



(a) Estimated attitude

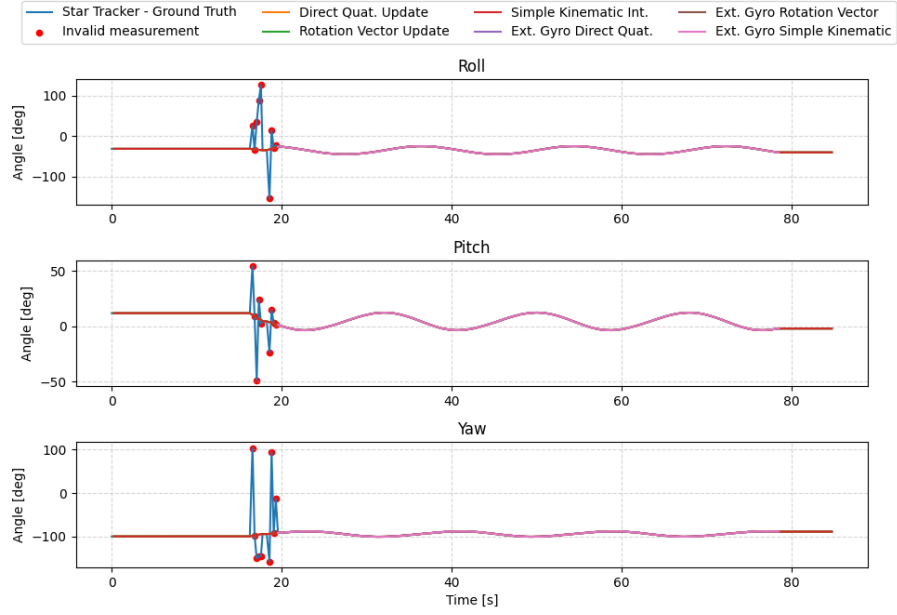


(b) Accumulated error (regular updates)

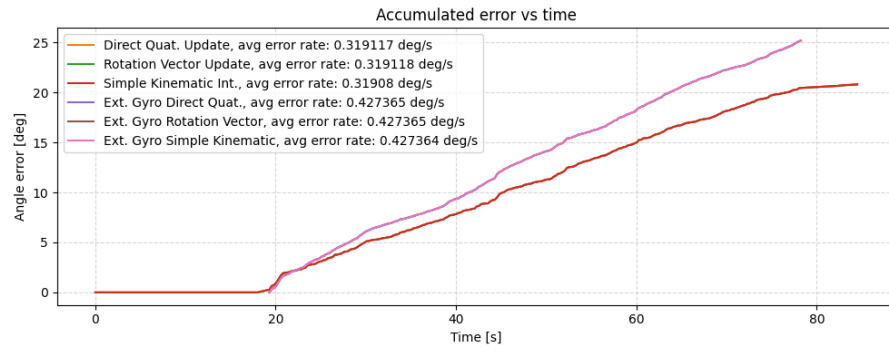


(c) Accumulated error (valid-only updates)

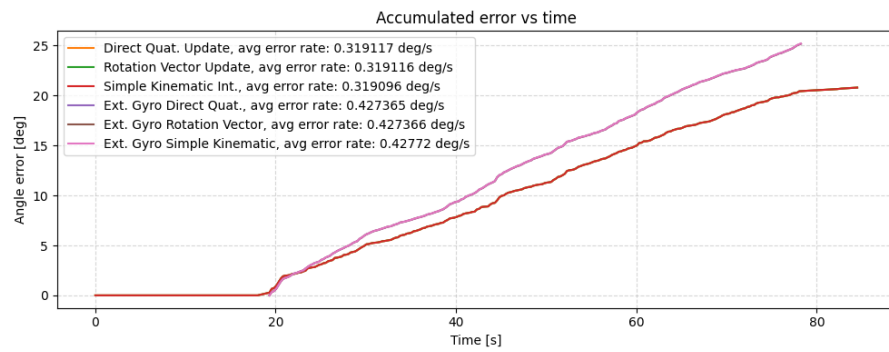
Figure D.2: Coning motion with $\alpha = 20^\circ$ and $\omega = 1$ rad/s.



(a) Estimated attitude

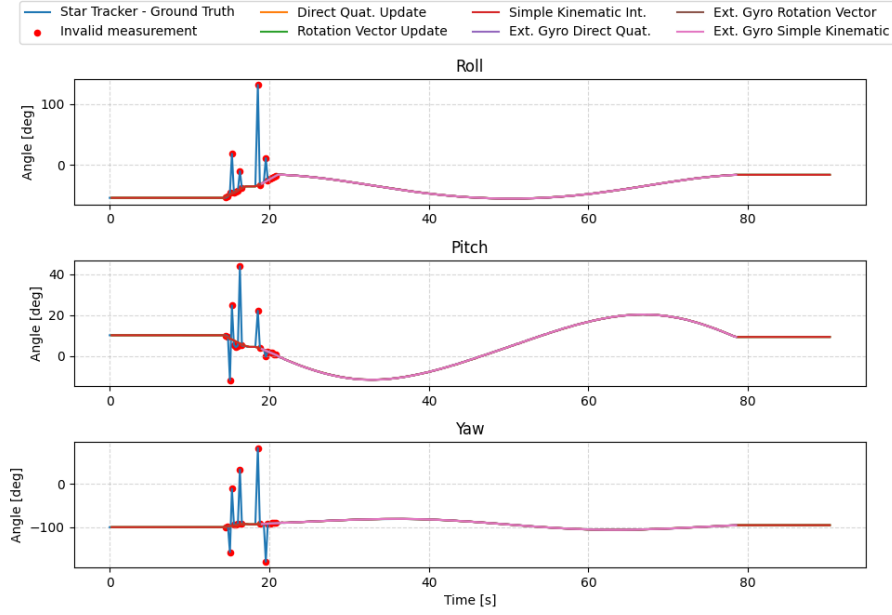


(b) Accumulated error (regular updates)

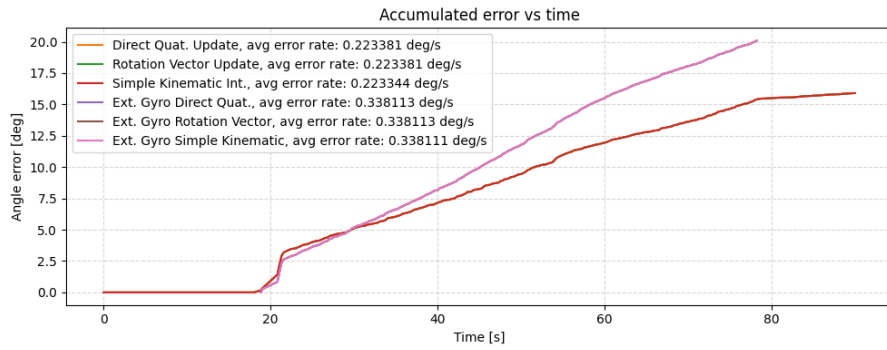


(c) Accumulated error (valid-only updates)

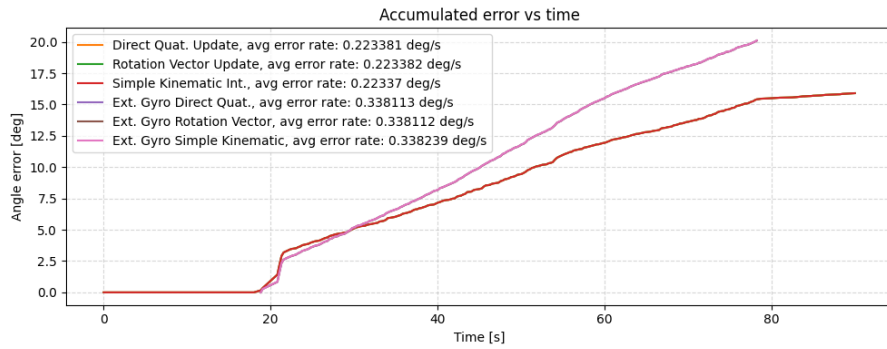
Figure D.3: Coning motion with $\alpha = 10^\circ$ and $\omega = 0.3$ rad/s.



(a) Estimated attitude



(b) Accumulated error (regular updates)



(c) Accumulated error (valid-only updates)

Figure D.4: Coning motion with $\alpha = 15^\circ$ and $\omega = 0.1$ rad/s.

E Use of Generative AI

In accordance with DTU guidelines on transparency in the use of generative AI, this appendix documents how such tools were used during this project.

The generative AI tool used was ChatGPT from OpenAI. It served as a support tool during the literature review process, for routine programming tasks such as debugging and plotting, formatting, and for refinement of language and improvement of academic phrasing during the writing process.

The tool was not used for algorithm design, generation of experimental results, derivation of conclusions, or development of core methodological content. All algorithmic descriptions, software implementations, experiment design, data collection, analysis, results, and conclusions remain the responsibility of the author. All AI generated content was critically reviewed and revised where necessary.

The following examples illustrate representative uses of the tool.

Language Optimization

Prompt: “Validate and rewrite the following paragraph in an academic tone suitable for a master’s thesis: This thesis is split in the following way. Section 2 describes the mathematical representation of attitude. Section 3 describes the commonly used sensors on spacecraft for attitude determination. Section 5 discusses attitude kinematics, defines the coning motion and describes different kinematic integration methods.”

Output: “This thesis is structured as follows. Section 2 introduces the mathematical representations of spacecraft attitude. Section 3 provides an overview of commonly used attitude sensors. Section 5 presents the kinematic formulation of attitude propagation, introduces coning motion, and describes the considered integration methods.”

Programming Assistance

Prompt: A plotting function was provided, and the following question was asked:

“I have six measurements and would like to insert a vertical dashed line between the first three and the last three bars.”

Output: The tool suggested the following:

“Add the following line after `ax.bar(...)`:

```
ax.axvline(2.5, linestyle="--", color="k", linewidth=1)”
```

Technical
University of
Denmark

Elektrovej, Buildings 327+328
2800 Kgs. Lyngby
Tlf. 4525 1700

www.space.dtu.dk